

大营销面试题梳理

DDD设计方面

大厂设计思路

1. 使用 DDD 做这个项目时，都运用了 DDD 哪些知识（领域模型和DDD架构的理解）
2. 抽奖流程中，哪些被定义为值对象，哪些被定义为实体对象（战术设计）
3. 访问数据层的依赖倒置 是怎样设计的
4. 聚合根的设计原则？
5. 一系列问题
6. 策略领域和奖品领域为什么要划分开

业务设计

1. 看到你简历上说把抽奖划分为抽奖前、中、后，三个动作。请具体结合场景讲解下，为什么这样设计
2. 抽奖也是一种瞬时峰值很高的业务场景，那么对于抽中奖品后的库存扣减是怎么做的？（最关键的技术点...
3. 库存的扣减是通过 Redis 滑块锁实现的👍，那么最终同步库是怎么做的，怎么降低对数据库的压力的？ ...
 - 3.1. 技术方案设计（RocketMQ延时消息+ xxl-job异步更新奖品库存）
4. 在项目中你提到了可以支持不同场景的抽奖诉求，比如；多少积分后可以抽奖一个固定范围的奖品，或者...
5. 库存扣减（最核心）
 - 5.1. 我的理解
 - 5.2. 对于恢复库存的理解
 - 5.3. 扣减库存的两种技术方案
 - 5.3.1. 不需要动态添加库存的场景
 - 5.3.2. 需要动态添加库存的场景（运营操作）
 - 5.3.2.1. 运营补库存的操作怎么做
6. 如果在系统运行过程中还要加库存呢，怎么办呢，如何保证redis与数据库的一致性
7. 当库存扣减时，redis中的库存扣减完成了，如果此时系统挂掉了，往mysql同步数据失败了，怎么办？ ...
8. 量化规则引擎是一个组件，如果有一个新的业务进来，如何复用？它的复用性体现在哪？能否支持风控可A...
9. 业务指标：拉新率 促活率
10. 扣库存的时候，如果库存在缓存扣成功了，但是消息队列没发送成功怎么办？
11. 规则引擎的设计目的
12. 抽奖算法如何提供O(1)时间复杂度，提高抽奖效率？

13. 如何对所有分布式节点的应用，活动信息本地内存更新？
14. 如果把redis中 滑块锁过期时间设置为活动过期时间的时候，如果活动时间很长导致滑块锁过多怎么解决
15. 15.抽奖前需要先创建一个抽奖订单（创建抽奖单可能扣减账户次数额度），然后真正执行抽奖。「抽...
16. 多个用户同时进行权重抽奖时，如何保证这些用户都抽到奖？
17. 在抽奖中后环节时，若此时对库存进行了修改，如何去更新缓存和数据库的数据？
18. 对sku下单时，扣减sku库存是先扣redis缓存 然后发MQ消息，通过一个定时任务缓慢更新数据库保证数...
19. 关于奖品库存扣减，假设库存是三个，若给一个用户发了5个奖品，如何处理？
20. 如果用户签到两次，如何防止获得多次抽奖充值？
21. incr防止超卖和decr防止超卖代码上如何实现，在业务上来讲有什么差别

设计模式（可以写进去）

1. 是什么场景下使用了责任链模式，什么场景使用了组合模式，为什么？
2. 接口的单一职责设计
 - 2.1. 抽奖策略接口
 - 2.2. 奖品信息接口
 - 2.3. 库存处理接口

技术问题

1. 为什么使用分库分表，做了分库分表数据聚合查询如何处理？
 - 1.1. 数据量也不大，为什么还要做分库分表
2. 方案设计：基于库表分段扫描和数据Redis预热，优化分布式延迟任务触达时效性
 - 2.1. 任务表设计
 - 2.2. 低延迟方式
 - 2.2.1. Redis 消费队列
3. Canal技术方案
4. 为什么不根据时间来分库分表，而是根据人员ID来分库分表，后期数据量大怎么扩容？
5. 分库分表怎么根据非分片键查询？
6. 你依赖于缓存，那假设Redis全down机了怎么办（虽然概率比较小），虽然有故障恢复，但是故障恢复有...
7. 生产者可能多次发送同一个MQ，怎么保证奖品不会超发？
8. 分库分表怎么让任务扫描到指定的库表
9. 如果在多机部署的情况下，是不是每台机器都会有这个定时任务，如果它们都捞到同一条发送失败的消息...
10. 为什么不直接使用内存作为缓存而引入Redis
11. 用redis 的set nx实现分布锁，为什么用set nx可以实现分布锁锁，set nx不是单节点的吗？

生产问题（力求真实）

思考

并发用户、RPS、TPS的解读

1. 你的抽奖接口响应时间是多少
2. 这个项目的业务数据，例如用户量、活跃用户量，以及核心业务数据，例如订单系统的下单量等.? 讲述...
3. 如果抽奖系统是1500qps，接口响应300ms，然后每秒的并发量是450个，然后面试官问参与人数的时...
4. 如果让你来评估项目的QPS的话，你会用什么方式来评估？（补充：不要做压测，就通过现在的设计以及...
5. 抽奖服务量化人群指标是什么？拉新 留存 促活吗 感觉这些数据好难算 面试官问留存率 拉新成本 我都不...

开放式问题

你在做项目中，什么问题难住你的时间最长，为什么？

这个营销系统是之前就有，你去接手了。还是你从0到1 构建的。

项目的安全性如何保证？

关于抽奖系统是不是高并发，与传统秒杀系统的区别与联系？

系统的数据怎么判定是否该放入缓存中(redis),判定的具体衡量标准是什么？

QPS与TPS的理解？

大厂面试题

美团】1. 领域模型是怎么设计的，抽奖过程是什么样，DDD四层架构和职责，以及为什么要这么设计？少...

【淘天】2. 对于数据库和redis的一致性怎么解决(关键)

【其他】3. 什么情况下使用DDD架构，什么情况下使用mvc架构？

【其他】4. 设计模式带来了什么好处？举个例子

【其他】5. 最近大营销抽奖项目被问到抽奖项目按道理应该不是一个高并发的过程，不是想那种抢优惠券...

【其他】6. 抽奖算法提问

【携程】7. 我跟他说用户在抽奖系统用积分兑换抽奖机会，我方会向mq发消息，积分微服务那边（不是...

【其他】8. 关于上下游交互的问题

【网易】9. 在抽奖过程时，需要预热处理，可是不可能让用户点击活动装配呀，这个问题你怎么解决？

【网易】10. 在抽奖的结果中，如果一个用户在前端抽到了一个奖品，对用户是可见的。可是在扣减库存...

【其他】11. 既然为了想承受更高并发使用redis做库存扣减，那生成奖品id后续要等中奖订单入库才能给...

【其他】12. 你说你的系统压测指标是3000QPS（简历上我写了3000QPS压测指标），你认为你做的是...

扣库存（最难点）

本篇主要内容为大营销的技术方案的设计和权衡，重点通过同学们在面试中遇到的真题进行分析，并辅助以AI智能体帮助思考面试官会问到的题目。记住一个原则，抓住业务和技术方案的重点，可以把订单

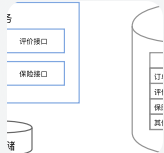
项目的方案拿过来结合业务去说。

架构：DDD架构 微服务设计

技术：Spring Boot / MyBatis / Dubbo / Nacos / Redis /ES/Sentinel /API 网关/MySQL

DDD设计方面

大厂设计思路



领域驱动设计在互联网业务开发中的实践

以美团外卖营销抽奖平台为例，讲述如何在互联网业务系统中实践领域驱动设计。

美团技术团队

1. 使用 DDD 做这个项目时，都运用了 DDD 哪些知识（领域模型和DDD架构的理解）

DD 是一种软件设计方法，软件设计方法涵盖了范式、模型、框架、方法论等内容，而 DDD 的很多规范约定，都是为了提高工程交付质量。如几个很重要的知识点；框架分层结构、领域、实体、聚合、值对象、依赖倒置等。它所有的手段，都希望以一个功能逻辑的实现为聚合，将功能所需的对象、接口、逻辑，按照领域划分到自己的领域内。

就像在这个项目中，我负责实现的抽奖中的**策略**，就是一个独立的领域模型。在这个领域中我需要提供策略的装载、随机数算法计算、抽奖模板调用（含责任链和规则树）功能，这样一个领域就像划分好的一个独立个体，它拥有属于它的对象信息（实体、值对象、聚合），当需要使用数据库资源、缓存资源，以及外部接口资源的时候，都通过依赖倒置进行调用。也就是说，**我的领域不做其他模块的引入，而是领域只负责业务功能实现，所需的所有数据，则有外部接口通过依赖倒置提供。**

2. 抽奖流程中，哪些被定义为值对象，哪些被定义为实体对象（战术设计）

详细设计

在 DDD 的规范定义中，值对象通常用于描述对象属性的值，**不具备唯一ID，不影响数据变化**。如；数据库中字段的枚举值、业务流程中属性对象。如抽奖流程中，RuleLimitTypeVO 规则限定方式的枚举值对

象、还有 RuleTreeVO 规则树值对象。

而那些实体对象，则具备唯一ID，会影响到最后的写库动作。如：抽奖发起实体、奖品信息实体对象。并且我们可以把一些和实体对象相关的功能聚合到对象内，这样的通用性会更好，避免所有调用方都需要自己编写逻辑。

3. 访问数据层的依赖倒置 是怎样设计的

DDD 中的依赖倒置是一个非常好的设计，尤其是与 MVC 结构对比的时候，MVC 的贫血模型结构设计，数据库持久化对象，很容易被当做业务对象使用，这样后期非常难维护。

但在 DDD 的分层结构用，是以 domain 领域实现为核心，一个 domain 领域下所需的外部服务，都由领域层定义接口，让基础层做具体实现。而数据库持久化操作，定义的 PO 对象，就被这样的方式被限定在基础层了，外部是没法引入使用的，也就天然的防止了数据库持久化对象进入业务中。

4. 聚合根的设计原则？

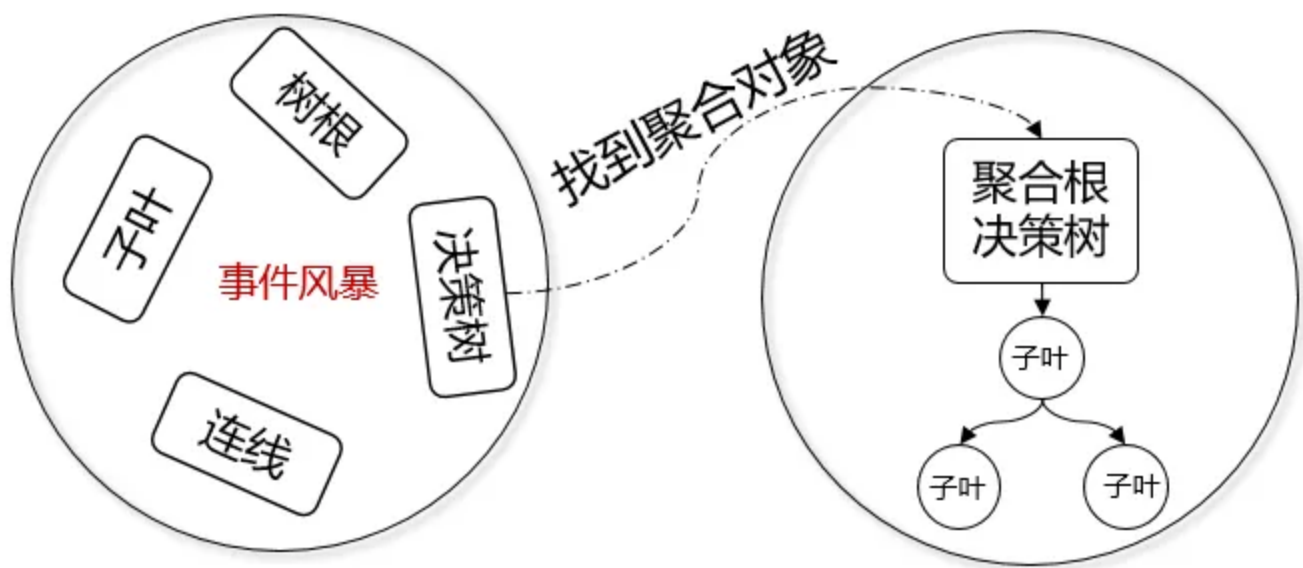
以活动入口的规则引擎举例，事件风暴出来的实体对象：树根、子叶、连线，而聚合根是一个整个决策树的描述。对于这块的内容，如果是理论，还有一些规则：

聚合设计的尽量小：如果聚合设计的过大，内部还有大量的实体和值对象，管理会比较复杂，高频操作会有并发和数据库锁冲突的问题，导致系统可用性降低；聚合设计的足够小，也就降低了复杂度，可复用性也更高，降低了后期重构复杂聚合的成本；

聚合应该高内聚：封装的是真正的不变的领域对象，内部的实体和值对象按照固定的规则运行，实现数据的一致性，边界外的任何东西都于该聚合无关。

使用最终一致性：聚合内部数据强一致性，聚合之间数据最终一致性，在一次事务中最多只修改一个聚合的数据状态，如果在一次事务中涉及修改多个聚合的状态，应该使用领域事件的方式来异步的实现最终一致性，实现聚合之间的解耦；

在应用层实现跨聚合的调用(领域编排)：实现微服务内部聚合之间的解耦，以为为未来以聚合为单位的拆分和组合，应该避免跨聚合的的领域服务调用和数据表关联；



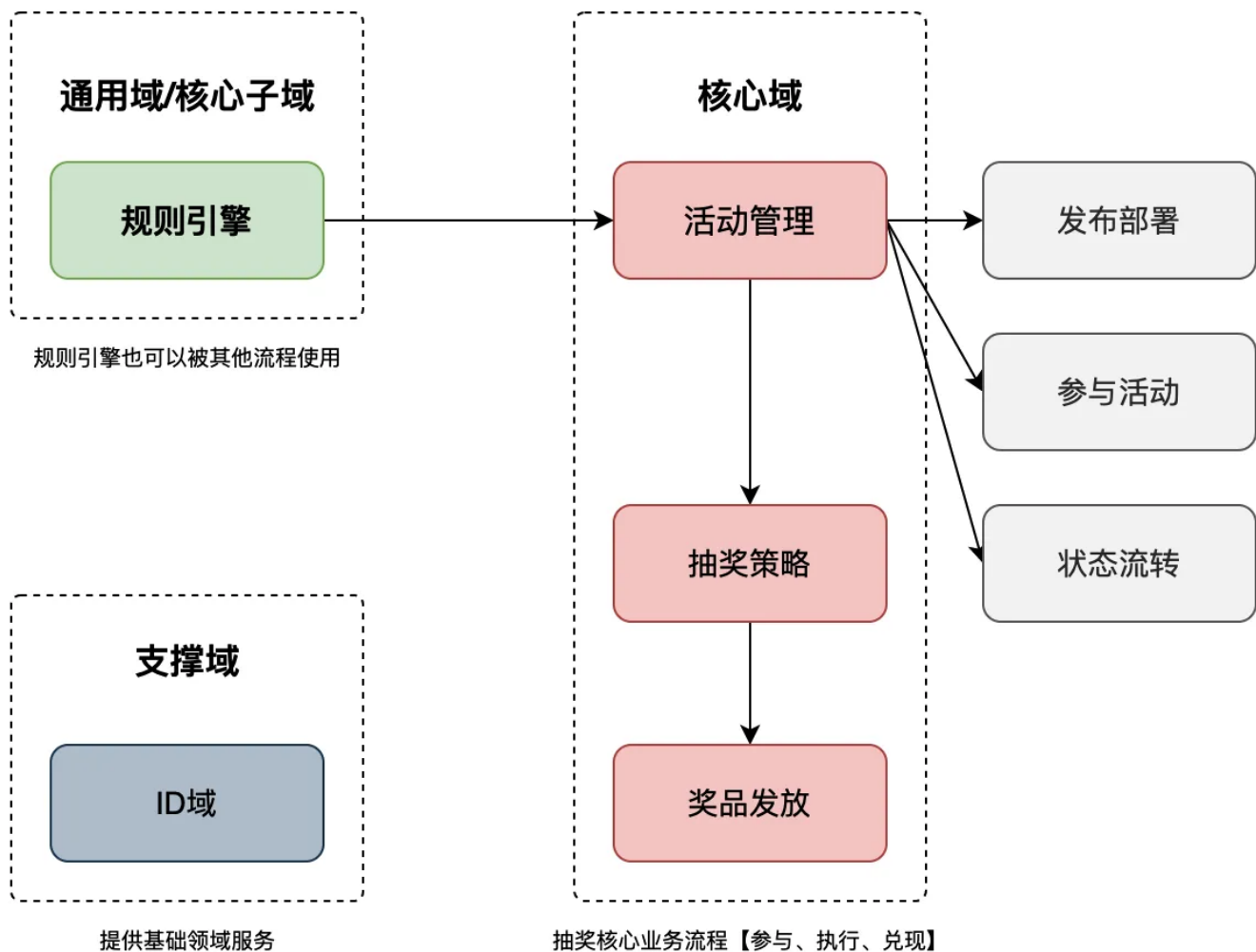
5. 一系列问题

1.你的DDD是怎么分层的？ 2.核心域的主要职责有哪些？ 3.核心域这边有哪些实体，哪些聚合？ 4.领域层有一个原则，就是尽量保证领域层的一个纯粹，那你如何保证领域层的一个纯粹性？ 5.防腐层的职责是什么？

1. 课程里做系统前，先讲了DDD分层；应用层、通用层、领域层、基础层、接口层、RPC定义层；第02节：搭建DDD四层架构 | bugstack 虫洞栈

2、3. 核心域主要职责；<https://t.zsxq.com/0bz8st5wJ>

1. 不同视角下其实解读为不同的域，其实也都可以【你这样列举也ok】。一般我们把主线流程成为核心领域，用于支撑主线流程的算作支撑领域或者核心子域。
2. 那么我们现在以抽奖为整个路线看，需要有3个步骤；参与、执行、兑现。也就是对应的活动、抽奖、奖品。而规则引擎其实没有也能完成抽奖，并且规则引擎也可以适用于其他模块下，所以它可以被看做是通用域/核心子领域。



4. 不只是领域层也包括其他层设计都要遵守软件的设计的第一原则；康威定律，分治、抽象和知识，只有把问题拆解的足够清晰，才越容易被处理。这包括；

4.1 遵循单一职责原则：确保每个类和方法都只关注一个特定的领域概念或逻辑，而不会处理与领域无关的内容。这有助于避免将非领域相关的逻辑混入到领域层中。

4.2 封装业务逻辑：确保所有业务逻辑都封装在领域层内部，以确保其他层不会直接访问领域逻辑。这有助于保护领域层的纯粹性，并使其更容易维护和扩展。

4.3 避免与其他层的交互：确保领域层仅与应用程序的其他领域层交互，而不与数据访问层或表示层等其他层直接交互。这有助于确保领域层只包含业务逻辑，并使其更易于理解和测试。

5. 防腐层的职责包括以下几个方面：

5.1 封装外部系统的接口和数据结构。外部系统通常使用自己的数据结构和接口，与领域层的模型不匹配。防腐层的职责是将外部系统的接口和数据结构进行封装，转化为领域层内部的数据结构和接口，使得外部系统对领域层不产生直接影响。

5.2 转化数据格式。在将外部系统的数据转换为领域层的数据时，防腐层需要对数据格式进行转化，使其与领域层的数据格式相匹配。例如，外部系统可能使用不同的日期格式，需要将其转化为领域层使用

的日期格式。

5.3 翻译领域语言。不同的系统使用不同的语言和术语来描述相同的概念。防腐层的职责是将外部系统的语言和术语翻译为领域层的语言和术语，以保持领域层的一致性。

5.4 确保数据一致性。在与外部系统进行数据交互时，防腐层需要确保数据的一致性。例如，在将数据从领域层传输到外部系统时，防腐层需要将数据进行验证和校验，以确保数据的准确性和完整性。

6. 策略领域和奖品领域为什么要划分开

1. 这部分要从建模讲，最开似的系统建模是整理出用户用例图，根据用例图梳理四色建模的领域事件。[四色建模过程\(opens new window\)](#)
2. 在领域事件脑暴完成，之后就是识别领域角色和对象，这个过程会显而易见的发现有**抽奖领域**、**发奖领域**。他们可以作为解耦设计，独立使用。因为抽奖不一定发奖，抽奖可以独立提供算法结果，由外部其他系统使用。发奖也可以除了抽奖的发奖，还有积分兑换的发奖等。如果抽奖和发奖合并，那么外部调用就会不那么清晰

业务设计

1. 看到你简历上说把抽奖划分为抽奖前、中、后，三个动作。请具体结合场景讲解下，为什么这样设计

这个的设计得益于在 Spring/MyBatis 框架源码的学习，在源码中经常会出现对一个流程进行拆分解耦，流程可扩展的点，如 Spring 是 Bean 对象的拆解，MyBatis 是会话流程的拆解。

所以在设计大营销的抽奖模块时，对于需求中的各类功能点；**黑名单抽奖**、**权重抽奖**、**默认抽奖**、**抽奖N次解锁**、**兜底抽奖**等等情况，是可以拆解为抽奖前、中、后，3个行为动作的，基于这样的考虑后，就可以设计出非常容易扩展的松耦合结构。

2. 抽奖也是一种瞬时峰值很高的业务场景，那么对于抽中奖品后的库存扣减是怎么做的？（最关键的技术点）

关于库存的扣减，是一个非常重要的流程。尤其是这种单独资源竞争的场景，如果设计的不好，很容易把服务打挂。

所以在这套系统设计中，为了避免库存扣减直接更新库表的行级锁，而导致大量的用户进行等待状态。所以把数据库表的库存同步到 Redis 缓存中，在通过 incr 扣减的方式进行消费，同时为了确保在**临界状**

态、库存恢复、异常处理等情况下不超卖，而对每一条产生从 incr 值，与抽奖的策略ID组合一个key，进行 setnx 加锁兜底，来保证不超卖。—— 这样的设计是颗粒度更小的锁方案设计，性能接近于无锁化。

3. 库存的扣减是通过 Redis 滑块锁实现的👍，那么最终同步库是怎么做的，怎么降低对数据库的压力的？（MySQL和数据库一致性的方案）

关于 redis 缓存和数据库表库存数据的流程，设计了异步更新，保持最终一致性的设计。在执行完库存的扣减操作后（在抽奖中规则树库存节点流程），发送一个扣减完成到 Redis 的异步队列（可以使用MQ+延迟消费），之后通过定时 Schedule Job 来消费队列。这样就可以控制效率速率，降低对数据库的压力。（因为我们不能 Redis 扣减的多快，就直接打到库表上，那样对数据库的压力依然很大，容易打挂）

过去，阿里云商用的 RocketMQ 支持任意时间延迟消息，但是开源 4.x 版本的不支持。只有默认 18 个 level 的延迟消息，分别是 1s 5s 10s 30s 1m 2m 3m 4m 5m 6m 7m 8m 9m 10m 20m 30m 1h 2h。然而，最新版本5.0中，RocketMQ引入了任意时间延迟消息的灵活性。它的实现是基于时间轮算法，并具有可调整的精度参数。

作者：schneiderlin

链接：<https://juejin.cn/post/7199523914821075004>

来源：稀土掘金

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

3.1. 技术方案设计（RocketMQ延时消息+ xxl-job异步更新奖品库存）

- RocketMQ延时消息，延时5S去消费。（甚至可以去改配置文件里的延时等级）可以改成3S。
- xxl-job 异步更新奖品库存

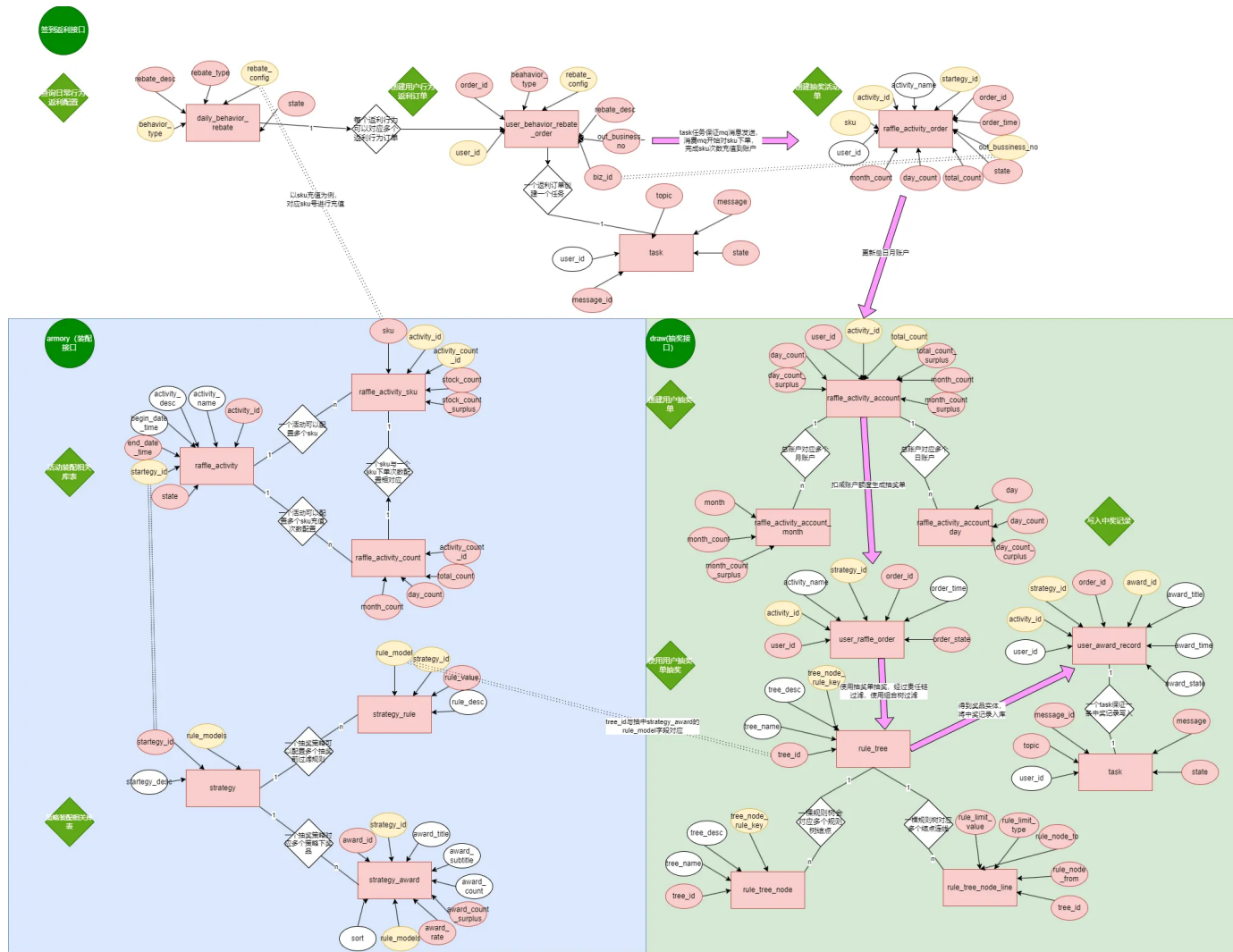
降低数据库压力；

如果还是存在奖品库存不一致的情况，会有一个兜底任务，在活动下线以后，扫描发奖单，去纠正数据库库存。

4. 在项目中你提到了可以支持不同场景的抽奖诉求，比如；多少积分后可以抽奖一个固定范围的奖品，或者抽奖n次后，才可以

中奖某个奖品。这部分你是怎么做的？库表怎么设计的？

库表包括；策略表、策略明细（库存、概率、规则key）、奖品表、规则表、规则树（3个表）



5. 库存扣减（最核心）

Redis库存：strategy_award_count_key_（策略id）_（奖品id） -> 库存

奖品库存在活动装配-策略装配时已经完成，等活动上线后，可以去扣减奖品库存。

（基于Redis扣减库存）

5.1. 我的理解

<https://wx.zsxq.com/group/48411118851818/topic/8852222545481582>

1. 其实本身原本的库存，扣减最早的方式是数据库行级锁，数据库扛不住。优化到redis缓存，使用了redis独占锁，但这样会出现排队问题，导致有库存，但吞吐量不佳。后来设计为**颗粒度更细的无锁化（乐观锁）设计**。incr/decr 是原子操作，基本也不会出问题，这个操作就有点像数据库计数，1、2、3、4... 而加锁类似写了流水记录。相对来说这样的方式更可靠。避免在；集群配置、网络、库存恢复、人工调整等场景时，导致 incr/decr 的值不对。**因为如果不加锁，这个时候不对，是不知道的。不知道就很可怕，哪怕有万分之一的概率，只要有足够多的用户参与，也会出问题。（尤其运营配置，运营是很多人，也有很多新人，实际公司里很多事故都是运营配置导致的，而且出问题的时候是一堆人围着一个运营，运营很慌，配置错的概率更大）**

2. 可以先调整数据库库存，之后使用 incrby 给缓存加库存。这个时候要注意使用 incr 和总量对比。如果有对库存的失败记录，可以用 incr 和 总量 + 恢复了对比。（恢复库存操作）

加锁，用于兜底；集群配置、网络、库存恢复、人工调整等。尽量避免出问题，加库存就是其中的一个。

恢复库存：Redis 库存 100 -> 80 扣减了 20 （99-80）之间这20个库存都不能扣减

- 库存扣减失败的两种情况
 - 如果可用库存小于0，那么就把redis奖品库存设置成0
 - 如果setnx失败，说明扣减了之前已经扣减过的库存，此时已经扣减过，不会重复扣减
- 只要库存扣减失败，就不会发送MQ消息异步去更新MySQL奖品的库存（没必要更新 库存扣减都失败了）同时后续活动完成，有定时任务去修复数据库中的奖品库存
- 同时库存扣减失败，发幸运奖（兜底奖品） 这个就是兜底的业务策略 可以配置

对于奖品的情况，可以少发，不能多发。因为奖品是固定的，少发一点没事，多发一点就是损失。

5.2. 对于恢复库存的理解

假设最开始库存最开始是100 扣了4次 99 98 97 96这几个加锁了 redis中的库存值目前是96 后来数据库库存更新到了120 那从96（应该有一个新的库存变量存96，原来的库存值应该不能变吧，因为后续还要从96开始decr）incr后再加锁，也会消费失败报异常吧，这几条加锁都过了之后，到后面继续加和总量比，如果后面的库存全都加完锁了，再从96开始decr吗？

- 奖品库存 incr 和奖品总库存（正常不会并发去更新，只有运营去补库存）比， 如果没超过，扣减就成功
- setnx 库存

5.3.2.1. 运营补库存的操作怎么做

- 更新数据库 策略奖品表的总库存

```
4
5 • select * from `strategy_award`;
```

ult Grid											
Filter Rows: Edit: Export/Import: Wrap Cell Content: 15											
id	strategy_id	award_id	award_title	award_subtitle	award_count	award_count_surplus	award_rate	rule_models	sort	create_time	update_time
1	100001	101	随机积分		80000	79998	0.3000	tree_luck_award	1	2023-12-09 09:38:31	2024-02-25 22:40:50
2	100001	102	5次使用		10000	9999	0.2000	tree_luck_award	2	2023-12-09 09:39:18	2024-02-25 22:40:55
3	100001	103	10次使用		5000	4998	0.2000	tree_luck_award	3	2023-12-09 09:42:36	2024-02-25 22:40:45
4	100001	104	20次使用		4000	3999	0.1000	tree_luck_award	4	2023-12-09 09:43:15	2024-02-25 22:40:00
5	100001	105	增加dall-e-2画图模型		600	600	0.1000	tree_luck_award	5	2023-12-09 09:43:47	2024-02-15 07:42:13
6	100001	106	增加dall-e-3画图模型		200	200	0.0500	tree_luck_award	6	2023-12-09 09:44:20	2024-02-15 07:42:14
7	100001	107	增加dall-e-3画图模型	抽奖1次后解锁	200	200	0.0400	tree_luck_award	7	2023-12-09 09:45:38	2024-02-15 07:42:17
8	100001	108	增加100次使用	抽奖2次后解锁	199	199	0.0099	tree_luck_award	8	2023-12-09 09:46:02	2024-02-15 07:42:21
9	100001	109	解锁全部模型	抽奖6次后解锁	1	1	0.0001	tree_luck_award	9	2023-12-09 09:46:39	2024-02-15 07:42:26
10	100002	101	随机积分		1	1	0.5000	tree_luck_award	1	2023-12-09 09:46:39	2024-02-15 07:42:29
11	100002	102	5次使用		1	1	0.1000	tree_luck_award	2	2023-12-09 09:46:39	2024-02-15 07:42:32
12	100002	106	增加dall-e-2画图模型		1	1	0.0100	tree_luck_award	3	2023-12-09 09:46:39	2024-02-15 07:42:35
13	100003	107	增加dall-e-3画图模型	抽奖1次后解锁	200	200	0.0400	tree_luck_award	7	2023-12-09 09:45:38	2024-02-15 07:42:38
14	100003	108	增加100次使用	抽奖2次后解锁	199	199	0.0099	tree_luck_award	8	2023-12-09 09:46:02	2024-02-15 07:42:41
15	100003	109	解锁全部模型	抽奖6次后解锁	1	1	0.0001	tree_luck_award	9	2023-12-09 09:46:39	2024-02-15 07:42:44
16	100004	109	解锁全部模型	抽奖6次后解锁	1	1	1.0000	tree_luck_award	9	2023-12-09 09:46:39	2024-02-15 07:42:46
17	100005	101	随机积分		80000	80000	0.0300	tree_luck_award	1	2023-12-09 09:38:31	2024-02-15 07:42:47
18	100005	102	随机积分		80000	80000	0.0300	tree_luck_award	1	2023-12-09 09:38:31	2024-02-15 07:42:48
19	100005	103	随机积分		80000	80000	0.0300	tree_luck_award	1	2023-12-09 09:38:31	2024-02-15 07:42:50
20	100005	104	随机积分		80000	80000	0.0300	tree_luck_award	1	2023-12-09 09:38:31	2024-02-15 07:42:51
21	100005	105	随机积分		80000	80000	0.0010	tree_luck_award	1	2023-12-09 09:38:31	2024-02-15 07:42:52
22	100006	101	随机积分		10000	9982	0.7900	tree_luck_award	1	2023-12-09 09:38:31	2024-06-22 11:53:30
23	100006	102	OpenAI会员卡		100	15	0.0300	tree_luck_award	2	2023-12-09 09:38:31	2024-06-22 09:59:55
24	100006	103	支付优惠券		100	38	0.0300	tree_luck_award	3	2023-12-09 09:38:31	2024-06-22 10:00:00
25	100006	104	小米台灯		100	28	0.0300	tree_luck_award	4	2023-12-09 09:38:31	2024-06-22 10:00:05
26	100006	105	小米su7周体验	抽奖3次后解锁	100	37	0.0300	tree_lock_3	5	2023-12-09 09:38:31	2024-06-22 09:59:45
27	100006	106	轻奢办公椅	抽奖2次后解锁	100	22	0.0300	tree_lock_2	6	2023-12-09 09:38:31	2024-06-22 10:00:25
28	100006	107	小霸王游戏机	抽奖1次后解锁	100	21	0.0300	tree_lock_1	7	2023-12-09 09:38:31	2024-06-22 10:00:10
29	100006	108	暴走玩偶		100	25	0.0003	tree_luck_award	8	2023-12-09 09:38:31	2024-08-24 16:18:40
30	10007	101	随机积分		1000000	1000000	0.1000	tree_luck_award	1	2024-10-06 12:00:40	2024-10-06 12:29:06
31	10007	110	1次使用次数		1000000	1000000	0.8000	tree_luck_award	2	2024-10-06 12:00:40	2024-10-19 09:42:56
32	10007	111	5次使用次数		1000000	1000000	0.0800	tree_luck_award	3	2024-10-06 12:00:40	2024-10-19 09:42:57
33	10007	112	10次使用次数		1000000	1000000	0.0100	tree_luck_award	4	2024-10-06 12:00:40	2024-10-19 09:42:58
34	10007	113	50次使用次数	抽奖3次后解锁	1000000	1000000	0.0002	tree_lock_3	5	2024-10-06 12:00:40	2024-10-19 15:05:33
35	10007	114	20次使用次数	抽奖2次后解锁	1000000	1000000	0.0040	tree_lock_2	6	2024-10-06 12:00:40	2024-10-19 15:05:28
36	10007	115	25次使用次数	抽奖1次后解锁	1000000	1000000	0.0008	tree_lock_1	7	2024-10-06 12:00:40	2024-10-19 15:05:30
37	10007	116	15次使用次数		1000000	1000000	0.0050	tree_luck_award	8	2024-10-06 12:00:40	2024-10-19 15:04:57
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- 发送MQ消息（事务消息）
- 后续消费 确保成功即可，出错，人工接入
- Redis设置总库存

○ 幂等性设计

- 加分布式锁，对于本次操作（分布式锁如果是zk，也可以）
- 总库存incrBy

6. 如果在系统运行过程中还要加库存呢，怎么办呢，如何保证redis与数据库的一致性

星球里的帖子有这个，需要先更新数据库库存，更新成功后，可以发mq，mq可以有补偿。确保一定会发mq。之后通过 setnx 给本次mq的流水id加锁，和库存 incrby 操作，一个 lua 脚本操作。这样可以保证唯一，如果失败了，也可以重试。

7. 当库存扣减时，redis中的库存扣减完成了，如果此时系统挂掉了，往mysql同步数据失败了，怎么办？ 库存扣减成功后，将中奖信息和task记录写入mysql失败，如何处理？

就是崩溃了，都不行了。怎么办的过程。

1. 服务，降级，挂挡板。暂时不对外，外部的活动页导流到其他模块。
2. 根据抽奖单记录校准库存，之后重新更新 Redis 库存。
3. 全部恢复后，重新对外活动。

8. 量化规则引擎是一个组件，如果有一个新的业务进来，如何复用？它的复用性体现在哪？能否支持风控可A/Btest需求？

1. 量化规则引擎的组件设计，本身就是一个独立的领域服务。但什么最开始没有给按照一个单独的系统它拆成独立的微服务呢？因为我们目前的实际业务场景体量还没有那么大，不太适合扩大维护成本，但又考虑将来可能会有其他业务也需要同类的模块，所以以独立的领域进行设计，哪怕以后真的有更大的场景需要使用，也能更加方便的拆分出去独立部署。
2. 它的复用性体现在，它是将同类业务场景中的共性需求，凝练成通用的业务组件，而不是把业务逻辑和功能性服务捆绑，所以在量化规则的库表中配置上相应的渠道和要决策的子节点，再通过子节点的决策树配置，就可以被新的业务场景使用。因为它是规则引擎结构设计，所以灵活性很好，复用性高。
3. A/B Test 本身就可以做为决策树中的节点果实来配置，对AB用户发放不同的策略结果。所以结合风控提供的数据，作为一个逻辑节点使用也是可以的。并且换个节点可以配置到不同的决策树中。
4. 另外决策树也可以进行连接，也就是扩展决策树的果实类型，不是发放结果，而是发放下一个决策树的ID，那么这样还可以把风控作为一个单独的共用决策树使用。其他需要使用风控模型的决策树，配置上即可。

9. 业务指标：拉新率 促活率

找美团的一些指标。

10. 扣库存的时候，如果库存在缓存扣成功了，但是消息队列没发送成功怎么办？

首先核心原则是保证不超卖，如果发送MQ失败了，那么有补充校验的任务系统会扫码抽奖时所产生的订单状态。补发MQ消息。（BCP巡检服务）

11. 规则引擎的设计目的

1. 面试官您好。关于规则引擎这块，首先；这是一个基于降低重复编码和提高可维护性的，并需要符合当前项目诉求的，同时不过渡的设计和减少运维成本的前提下，在技术调研后所做的微型规则引擎设计实现。
2. 此规则引擎的主要作用是解决，抽奖业务场景中对个性化运营诉求的处理，如；人群身份标签、交易记录、活动资格、授信状态等规则的可配置化的交叉使用。
3. 所以基于这样的情况，此规则引擎的设计是一个二叉树判断，实现手段运用到了组合模式、工厂模式等。并为了便于维护和使用，进行了库表对二叉树的抽象设计，树根、节点、子叶，映射为二叉树编码的相关属性信息。同时，也可以基于这样的库表做前端页面的托拉拽配置操作，降低运营成本。
4. 最后，其实动态的规则引擎配置，其实放大了看就是 BPMN + Drools + Groovy，的一个低代码实现框架。综上，面试官这个就是我在做规则引擎设计的一些思考、调研和落地。如果以后咱公司有其他更大的场景，我也可以扩展为 Rete 算法和低代码方案进行架构落地实现

12. 抽奖算法如何提供O(1)时间复杂度，提高抽奖效率？

在大营销系统中，运营人员配置好抽奖活动后，开始上线对外后，会进行数据的预热数据。这个预热的过程会把活动信息、策略信息、库存信息都存储到 Redis 里进行使用。

而抽奖的策略就是记录了一个策略下N个奖品的概率，将概率转换为对应的整数数量，写入到缓存中。那么在抽奖的时候就按照整数数量生成随机数来抽奖。这样用空间换时间的效率是非常高的。

13. 如何对所有分布式节点的应用，活动信息本地内存更新？

通常我们会有诉求在不重启系统的时候，就要动态变更所有分布式应用节点中某个属性的值，如开关、缓存、调试日志开启/关闭、熔断、限流、或者抽奖黑名单以及概率等。这些东西通常不是 Redis 存储，而是应用中具体字段的属性值，这样效率更高。

而这个操作需要使用到类似于 Zookeeper 组件的临时节点监听，动态变更字段值。

14. 如果把redis中 滑块锁过期时间设置为活动过期时间的时候，如果活动时间很长导致滑块锁过多怎么解决

可以考虑给活动库存的锁的key上年月日，每个日的key，明天就重新从新的key开始了。之后这样就比较好较短时间存储了。

Redis集群方式用Cluster，分散到不同的节点上。

15. 15.抽奖前需要先创建一个抽奖订单（创建抽奖单可能扣减账户次数额度），然后真正执行抽奖。「抽奖订单」的设计，相对于「直接扣减账户中的抽奖次数余额」，能够解决什么实际问题？

```

1
2      // 2. 查询未被使用的活动参与订单记录
3      UserRaffleOrderEntity userRaffleOrderEntity = activityRepository.q
queryNoUsedRaffleOrder(partakeRaffleActivityEntity);
4      if (null != userRaffleOrderEntity) {
5          log.info("创建参与活动订单存在 userId:{} activityId:{} userRaffle
OrderEntity:{}", userId, activityId, JSON.toJSONString(userRaffleOrderEnti
ty));
6          return userRaffleOrderEntity;
7      }
8
9      // 3. 额度账户过滤&返回账户构建对象
10     CreatePartakeOrderAggregate createPartakeOrderAggregate = this.doF
ilterAccount(userId, activityId, currentDate);
11
12     // 4. 构建订单
13     UserRaffleOrderEntity userRaffleOrder = this.buildUserRaffleOrder
(userId, activityId, currentDate);
14
15     // 5. 填充抽奖单实体对象
16     createPartakeOrderAggregate.setUserRaffleOrderEntity(userRaffleOrd
er);
17
18     // 6. 保存聚合对象 - 一个领域内的一个聚合是一个事务操作
19     activityRepository.saveCreatePartakeOrderAggregate(createPartakeOr
derAggregate);
20     log.info("创建活动抽奖单完成 userId:{} activityId:{} orderId:{}", use
rId, activityId, userRaffleOrder.getOrderId());
21     // 7. 返回订单信息
22     return userRaffleOrder;

```

1. 抽奖单设计是一种行为记录的流水，有了抽奖单可以做幂等重试，如果后序流程失败了，抽奖单不会被消费，用户可以重新使用抽奖单抽奖。如果是系统随机发奖，还可以系统基于失败的抽奖单自己做重试补偿。这种思想来自于商城下单，有了订单之后才有订单的支付。
2. 订单的存在可以让流程有了暂停和继续的操作。如果只是直接扣减账户抽奖次数余额，就只会得到执行结果，那么都是即时状态，过后无法追查校准。

抽奖单创建：

1. 创建抽奖单
2. 扣减账户抽奖次数余额

16. 多个用户同时进行权重抽奖时，如何保证这些用户都抽到奖？

凡是能达到权重范围的用户，是运营提前做了预算库存的，所以一定是会给发奖的。

17. 在抽奖中后环节时，若此时对库存进行了修改，如何去更新缓存和数据库的数据？

一般来说，不太允许更新已经发布的活动的库存，容易造成客诉。所有的运营手段，库存的设置，都是提前申请审批的，不是随意加的。如果要加，可以先更新数据库，更新成功后，添加缓存库存即可。

18. 对sku下单时，扣减sku库存是先扣redis缓存 然后发MQ消息，通过一个定时任务缓慢更新数据库保证数据最终一致性。如果redis缓存扣减成功之后，redis就挂了，后面的MQ消息也没有发出去，那么重启redis之后 又从DB中重新导入数据，这样就导致了数据不一致，可能出现超卖的情况 怎么解决？

使用setnx做了兜底操作，就算获取了旧的库存数据，但之前的锁都在redis里，可以防止超卖。

另外xfg补充：一般真挂了，活动就加挡板了，不会对外了。等恢复了，要校准数据，之后才能对外的。另外 redis 挂了，集群都挂了，是重大事故，各个服务也都不可用了。这个时候不会对研发下手，要干的是redis运维。

19. 关于奖品库存扣减，假设库存是三个，若给一个用户发了5个奖品，如何处理？

库存扣减成功的准许发放相应的奖品，库存扣减失败的奖品（会有两个），走兜底奖励。每一个库存的扣减都有粒度很细的分布式锁兜底（setnx，decr操作）

20. 如果用户签到两次，如何防止获得多次抽奖充值？

根据当前日期作为业务字段，拼接上**用户名，返利类型，构成唯一ID**，保证用户一天只能签到一次。

(mysql底层是如何实现唯一键约束的？通过在需要唯一性约束的列上创建唯一索引来实现的。唯一索引使用B+树等数据结构，它可以快速检索并保证索引列的唯一性。)

21. incr防止超卖和decr防止超卖代码上如何实现，在业务上来讲有什么差别

incr防止超卖的实现：

– 从0开始累加，直到达到缓存中记录的库存个数，就不再卖出商品。 – 从库存开始累加-1，和decr差不多

使用场景：动态添加库存

decr防止超卖的实现： – 扣减库存，当库存扣减为-1，则说明库存不足

使用场景：库存活动开始时就已固定好，后续很难或者无法再改变

设计模式（可以写进去）

1. 模板方法模式
2. 责任链模式
3. 组合模式

1. 是什么场景下使用了责任链模式，什么场景使用了组合模式，为什么？

在设计完抽奖前、中、后，松耦合的结构模型后，对于**抽奖前要执行哪种抽奖，但单向选择问题**。所以这里使用了责任链模式，进行节点流程判断，从黑名单、权重，最后到默认，走一个单独的具体抽奖，所以使用责任链更为合适。

组合模式：之后是进入抽奖的中和后，这两部的流程是相对复杂的，**需要判断用户抽奖了几次，对于不同次会限定是否能获得某个奖品，同时还有库存的扣减，如果库存不足或者不满足n次抽奖得到某个奖品，则会进行兜底**。那么这就是一个树规则的交叉流程，所以会使用了组合模式构建一颗规则树，并通过数据库表的动态配置决定在抽奖前完成后，后续的流程要如何进行。

2. 接口的单一职责设计

单一职责原则的核心思想是，一个类应该只有一个引起它变化的原因。也就是说一个类应该只负责一项任务或功能，如果一个类承担了过多的职责，那么这个类就会变得复杂，难以维护和扩展。

这样的原则在一些需要长期使用、迭代、维护的功能设计上，是非常重要的。我们要尽可能的让大营销的抽奖领域模块具备独立性，所以要使用单一职责原则。在这个原则约束下，设计了3个接口类；**抽奖策略接口**、**奖品信息接口**、**库存处理接口**（异步扣减等），这样3个接口的设计，在将来需要扩展的时候，会非常容易。

2.1. 抽奖策略接口

```
1  /**
2   * @author Fuzhengwei bugstack.cn @小傅哥
3   * @description 抽奖策略接口
4   * @create 2024-01-06 09:19
5   */
6  public interface IRaffleStrategy {
7
8      /**
9       * 执行抽奖；用抽奖因子入参，执行抽奖计算，返回奖品信息
10      *
11      * @param raffleFactorEntity 抽奖因子实体对象，根据入参信息计算抽奖结果
12      * @return 抽奖的奖品
13      */
14      RaffleAwardEntity performRaffle(RaffleFactorEntity raffleFactorEntity);
15
16  }
```

2.2. 奖品信息接口

```
1  /**
2   * @author Fuzhengwei bugstack.cn @小傅哥
3   * @description 策略奖品接口
4   * @create 2024-02-14 16:44
5   */
6  public interface IRaffleAward {
7
8      /**
9       * 根据策略ID查询抽奖奖品列表配置
10      *
11      * @param strategyId 策略ID
12      * @return 奖品列表
13      */
14      List<StrategyAwardEntity> queryRaffleStrategyAwardList(Long strategyId);
15
16      /**
17       * 根据策略ID查询抽奖奖品列表配置
18       *
19       * @param activityId 策略ID
20       * @return 奖品列表
21       */
22      List<StrategyAwardEntity> queryRaffleStrategyAwardListByActivityId(Long activityId);
23
24      /**
25       * 查询有效活动的奖品配置
26       *
27       * @return 奖品配置列表
28       */
29      List<StrategyAwardStockKeyVO> queryOpenActivityStrategyAwardList();
30
31  }
```

2.3. 库存处理接口

```
1  /**
2   * @author Fuzhengwei bugstack.cn @小傅哥
3   * @description 抽奖库存相关服务，获取库存消耗队列
4   * @create 2024-02-09 12:17
5   */
6  public interface IRaffleStock {
7
8      /**
9       * 获取奖品库存消耗队列
10      *
11      * @return 奖品库存Key信息
12      * @throws InterruptedException 异常
13      */
14      StrategyAwardStockKeyVO takeQueueValue() throws InterruptedException;
15
16      /**
17       * 获取奖品库存消耗队列
18       *
19       * @return 奖品库存Key信息
20       * @throws InterruptedException 异常
21       */
22      StrategyAwardStockKeyVO takeQueueValue(Long strategyId, Integer awardId) throws InterruptedException;
23
24      /**
25       * 更新奖品库存消耗记录
26       *
27       * @param strategyId 策略ID
28       * @param awardId 奖品ID
29       */
30      void updateStrategyAwardStock(Long strategyId, Integer awardId);
31  }
32
33
```

技术问题

1. 为什么使用分库分表，做了分库分表数据聚合查询如何处理？

首先我们要知道，是**前期就做好分库分表方案**，还是后期在做系统重构分库分表和数据迁移哪个成本高。显然是后面的成本更高。

而互联网中大厂中，分库分表的架构设计都是非常熟练的，因为有成熟方案，所以前期就分库分表了。但，为了节省服务器空间。所以把分库分表的库，用服务器虚拟出来机器安装。这样即不过多的占用服务器资源，也方便后续数据量真的上来了，好拆分。

那么这里的分库分表后的数据怎么提供汇总、聚合的查询呢？

这里需要用到阿里的 canal 组件，基于 mysql 的 binlog 日志，把自己伪装为一个从数据库，通过 dump 交换完成数据的接收和处理。**最终把数据同步到 Elasticsearch 等文件服务中在提供聚合查询。对于需要实时的查询以及数据的处理**，还可以用到 Flink 方式进行流式计算

16库 64表 一个表 50万数据。

比如50万用户，10万活跃用户，怎么去搞呢

1.1. 数据量也不大，为什么还要做分库分表

分库分表基本是单表200万，才分。那么面试怎么说；

你们为什么分库分表？

1. 我们分库分表用的非常熟。但不能为了等到系统到了200万数据，才拆。那么工作量会非常大
2. 我们的做法是，因为有成熟方案，所以前期就分库分表了。但，为了解释服务器空间。所以把分库分表的库，用服务器虚拟出来机器安装。这样即不过多的占用服务器资源，也方便后续数据量真的上来了，好拆分。
3. 同时，抽奖系统，是**瞬时峰值较高的系统**，历史数据不一定多。所以我们希望，**用户可以快速的检索到个人数据，做最优响应**。因为大家都知道，抽奖这东西，push发完，基本就1~3分钟结束，10分钟人都没了。所以我们这也是做了分库分表的理由。

4库32表 500万行数据 可以支撑业务10年以上的发展。

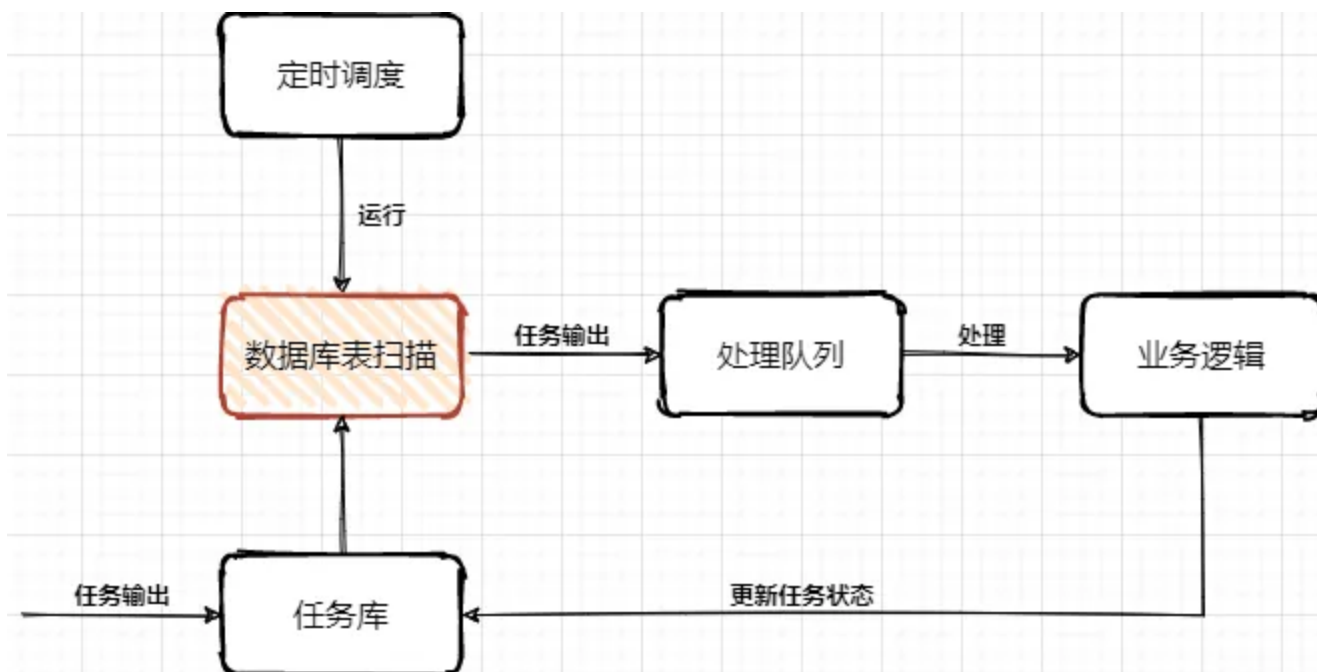
2. 方案设计：基于库表分段扫描和数据Redis预热，优化分布式延迟任务触达时效性



方案设计：基于库表分段扫描和数据Redis预热，优化分布式延迟任务触达时效性

包含: Java 基础, 面经手册, Netty4.x, 手写MyBatis, 用Java实现JVM, 重学Java设计模式, Spring...

小傅哥 bugstack 虫洞栈



痛点：

1. 海量数据规模较大的任务列表数据，在分库分表下该需要快速扫描。
2. 任务扫描服务与业务逻辑处理，耦合在一起，不具有通用性和复用性。
3. 细分任务体系有些是需要低延迟处理的，不能等待过长时间

由定时任务扫描任务库表，把即将达到超时时间的任务信息扫描到处理队列(内存/MQ消息)，再由业务系统进行处理任务，处理完成后更新库表中的任务状态。

2.1. 任务表设计

选择做一个通用的任务延时系统，各业务系统把需要被延时执行的动作提交到延时系统中，再有延时系统在指定时间下进行回调，回调的动作可以是接口或者MQ消息进行触达

:

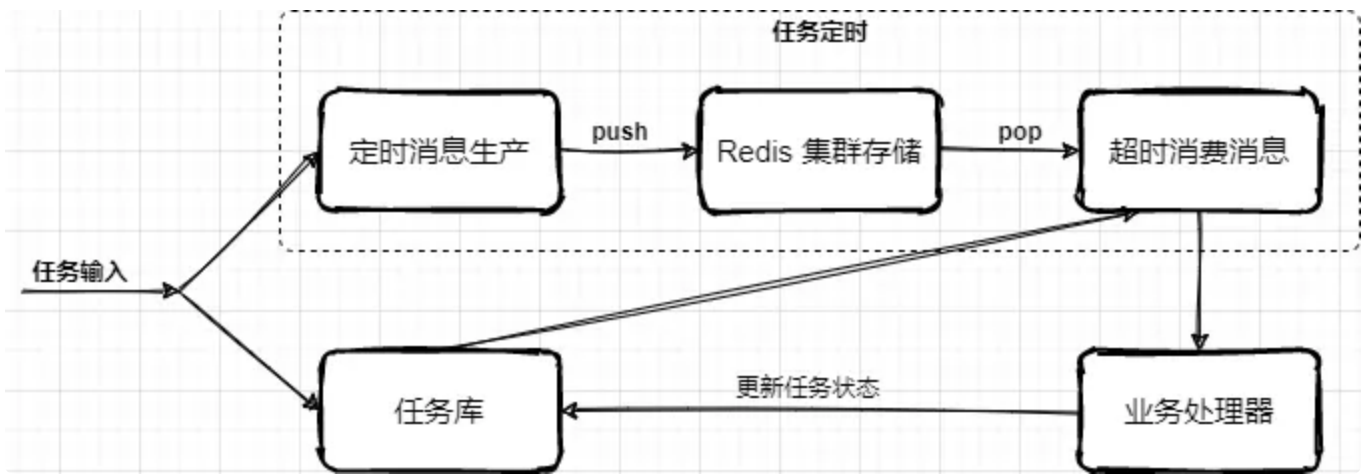
任务调度表_1001 {x个库y个表}							
ID	任务ID	业务ID	业务类型	状态	任务执行时间	扩展数据	门牌号
1	10001	101	订单	0	2022:02:14 00:00:00	情人节消费订单	1
2	10002	102	息费	1	2022:02:15 00:00:00	情人节订单息费	2
							3
							4
							5
							6
							7
							8
							9

分组扫描:
147、258、369

1. 抽取的任务调度表，主要是拿到什么任务，在什么时间发起动作，具体的动作处理仍交给业务工程处理。
2. 大批量的各自业务的任务进行集中处理，则需要设计一个分库分表，满足于后续业务体量的增长。
3. 门牌号设计，针对一张表的扫描，如果数据量较大，又不希望只是一个任务扫描一个表，可以多个任务扫描一个表，加到扫描的体量。这个时候就需要一个门牌号来隔离不同任务扫描的范围，避免扫描出重复的任务数据

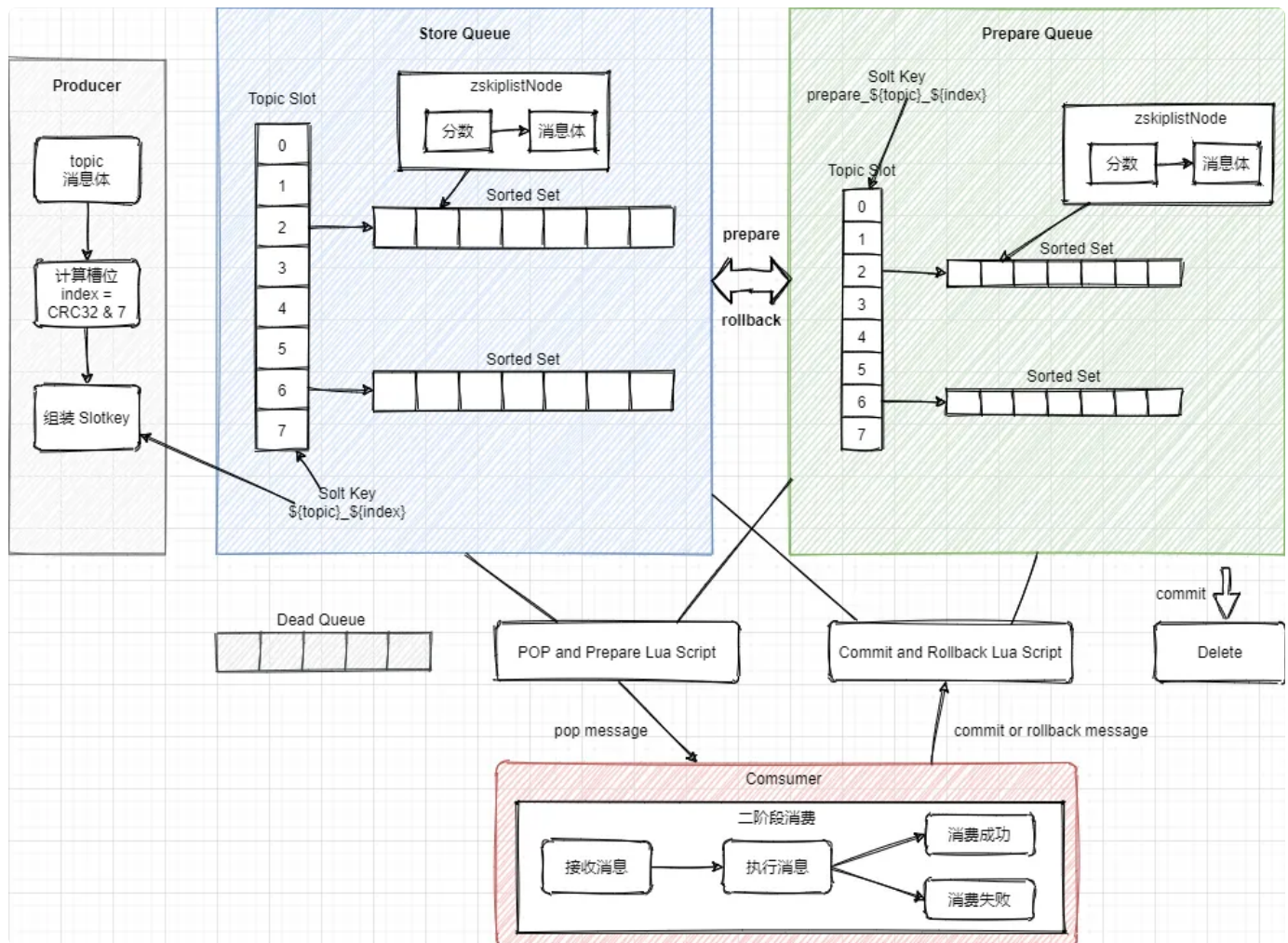
2.2. 低延迟方式

低延迟处理方案，是在任务表方式的基础上，新增加的时间把控处理。它可以把即将到期的前一段时间的任务，放置到 Redis 集群队里中，在消费的时候再从队列中 pop 出来，这样可以更快的接近任务的处理时效，避免因为扫库间隔较大延迟任务执行。



- 在接收业务系统提交进来的延迟任务时，按照执行时间的长短放置到任务库或者也同步到 Redis 集群中，一些执行时间较晚的任务则可以先放到任务库，再通过扫描的方式添加到超时任务执行队列中。
- 那么关于这块的设计核心在于 Redis 队列的使用，以及为了保证消费的可靠性需要引入二阶段消费、注册 ZK 注册中心至少保证一次消费的处理。

2.2.1. Redis 消费队列



- 按照消息体计算对应数据所属的槽位 $\text{index} = \text{CRC32} \& 7$
- StoreQueue 采用 Slot 按照 $\text{SlotKey} = \{\text{topic}\}_{\text{index}}$ 和 Sorted Set 的数据结构按执行任务分数排序，存放任务执行信息。定时消息将时间戳作为分数，消费时每次弹出分数小于当前时间戳的一个消息
- 为了保障每条消息至少可消费一次，消费者不是直接 pop 有序集中的元素，而是将元素从 StoreQueue 移动到 PrepareQueue 并返回消息给消费者。消费成功后再从 PrepareQueue 从删除，如果消费失败则从 PrepareQueue 重新移动到 StoreQueue，这样二阶段消费的方式进行处理。

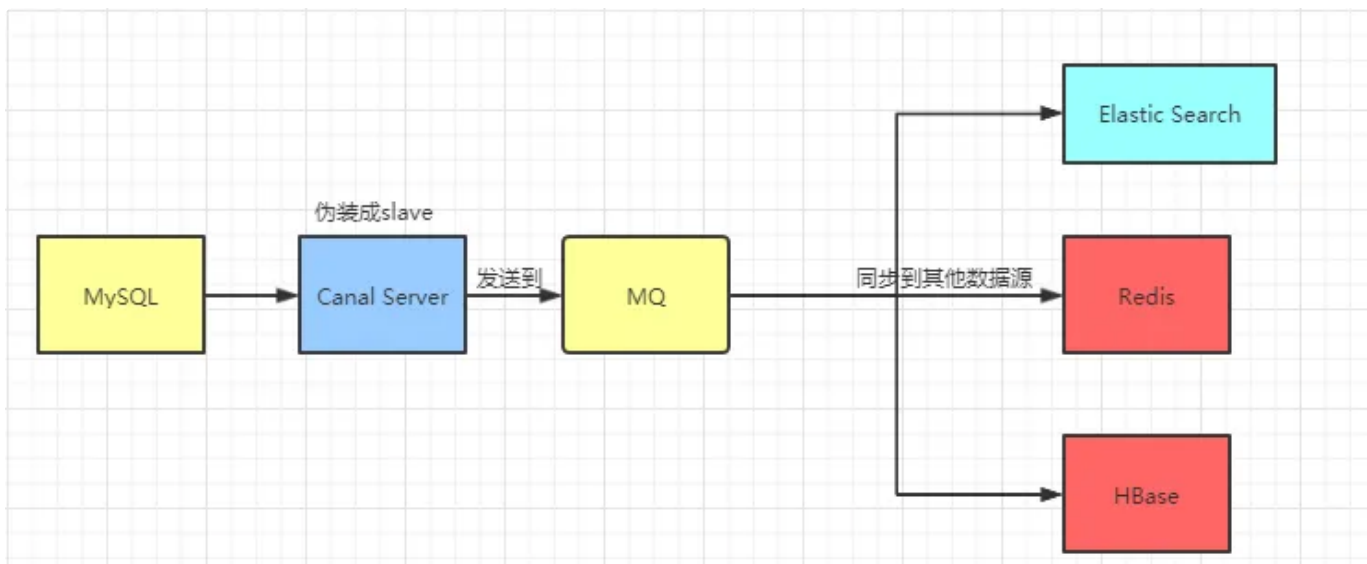


一种低延迟的超时中心实现方式-阿里云开发者社区

在很多产品中都存在生命周期相关的设计，时间节点到了之后需要做对应的事情。超时中心（TimeOut...

<https://developer.aliyun.com/article/784457>

3. Canal技术方案



ES 消费处理基于版本号

4. 为什么不根据时间来分库分表，而是根据人员ID来分库分表，后期数据量大怎么扩容？

1. 根据时间做，需要按照时间分片进行处理，比较适合处理冷数据。
2. 目前抽奖系统为实时业务数据，如果基于时间分片处理带来的问题是，当一个用户请求进来做业务，很难定位这个用户所属的库表，也包括一些基于人维度的事务处理。
3. 一般基于用户ID所做的分库分表，会根据业务体量的发展设定一个2-3年增量的分库分表规模。比如每个库32张表，分4个库。(PS：这里不要说就4个表、8个表，那你分表还有个P用)
4. 为了因分库分表所带来的服务器资源占用数量，会采用虚拟机操作，一般一台标C的物理机，进行1虚5的配置进行处理，这样在后续第一波业务爆发增长的时候，可以把虚拟机换为物理机，这个时候是不需要迁移数据的，成本较低。
5. 当后期数据量较大的时候，采用binlog+canal同步数据的方式进行扩容，并逐步把新数据写入到新库中，当两方数据库完全同步后，开始重启实例进行切换。当然这个成本和风险是会有有的，但随着我们的整个流程完整度的提高，在公司内部已经成为标准SOA作业。（分库分表扩容）
6. 当然我们还有一些额外的考虑，怎么做到自动扩展，针对这方面我们目前开始尝试给用户生成的ID中写入库表信息，这个信息是加密的，随着用户量的增多自动扩展数据库表，随着用户体量的增加，挂在后新的用户就可以注册到新库表了。另外，我们还思考，把数据库的路由前置到RPC层，解决数据库笛卡尔积交叉链接的问题，当然这是另外一个场景了，这里就不做过多的扩展了，面试官你看还有其他问题吗。

5. 分库分表怎么根据非分片键查询？

若问到以其他字段查询的问题，可以：

分库分表，uid做hash水平分表，若需要通过其他字段实时查询，例username，则可以有以下解决思路

用户端（实时）：

1. 可以做username到uid的映射，通过建立索引表，username定位到uid，可以将这份映射存入缓存。
2. 生成id，例如：uid=f(username)，缺点有id冲突。
3. 基因法，通过username抽取“基因”融入uid中

运营端：

1. 前后台分离架构，web/service/db分离，避免后台的查询影响前台
2. 数据冗余，宽表
3. 搜索引擎

6. 你依赖于缓存，那假设Redis全down机了怎么办（虽然概率比较小），虽然有故障恢复，但是故障恢复有可能会数据丢失？

Redis 宕机，触发挡板机制，活动暂时下线。数据恢复后在线上活动。但基本我们整个抽奖系统运行n年，100%交付需求，全年无此类重大事故发生。这也得益于我们所建立的 系统开发交付 SOP 标准化流程体系。

(活动先下线，走降级；定时任务去扫描抽奖单 更新库存等数据 等数据更新完毕后，再重新上线服务 Redis只是一个缓存，我们只要有手段保存MySQL数据恢复，那么Redis数据即可恢复)

数据不一致后，要先处理数据 等数据恢复了再说

7. 生产者可能多次发送同一个MQ，怎么保证奖品不会超发？

这是一个幂等的设计处理，MQ 的消息是必须含带具有唯一标识的业务ID的。比如订单ID、奖品ID、支付单ID、交易单ID、贷款单ID等等。接收MQ的系统，通过唯一ID业务，更新或者写库的时候可以保证幂等性。这样也就不会产生超发的可能。

8. 分库分表怎么让任务扫描到指定的库表

分库分表以后，需要扫描每个库表中的任务表，则需要手动设定具体要扫描的库和表。如果分库分表的数量比较多，可以用不同的任务配置扫描不同的库表方式来部署，这样可以提高扫描效率。

9. 如果在多机部署的情况下，是不是每台机器都会有这个定时任务，如果它们都捞到同一条发送失败的消息，会不会导致消息的重复发送？怎么避免？

2. 一个任务就是要有多机备份，避免一个挂了，就没有人执行了。之后这里的方案是加锁；
 - 2.1 设计一个抢占锁，多个任务抢占同一个锁，谁抢占到了，谁可以执行。
 - 2.2 如果抢占的执行失败了，删掉锁，重新执行。
 - 2.3 如果删锁失败，对于是谁抢占的，谁可以做重入锁，继续执行。
 - 2.4 锁有失效时间，如果抢占到的自己挂了，等待锁失效后，重新轮候抢占。

首先任务分片屏蔽这种情况；如果还是遇到这种情况加锁，再执行。（锁是有超时时间的，不会太短）

10. 为什么不直接使用内存作为缓存而引入Redis

1. 在营销复杂计算场景中，为了提高性能确实会用本地内存 + redis 缓存的方案。但本地内存会有一个问题，就是分布式架构下，在初始和变更数据，需要所有环境保持统一数据，并需要配有动态配置中心来通知更新。需要一定的维护成本。
2. 一般做这类的系统，以及配置类的，是会存redis一份，之后在拉取到本地内存一份。本地内存与redis 中数据进行版本校验和定期更新。这个是实际场景方案，所有面试官会追着你这样问。下次可以讲两套方案是一起使用的【不过最好最好案例，流程先跑通，避免解释不清。】
3. 对于2方案，结合 redis 也有发布订阅能力，可以发完成本地内存更新，这个在《动态线程池组件》中使用了。

11. 用redis 的set nx实现分布锁，为什么用set nx可以实现分布锁锁，set nx不是单节点的吗？

1. setnx是原子操作，确保只有一个客户端能成功获取锁。其余客户端全部失败。（什么是原子性？Redis的原子性操作指的是在执行过程中不会被其他操作打断，即操作要么完全执行成功，要么完全不执行，从而保证数据的一致性和可靠性。）
2. 超时机制，set nx 可以设置额外参数 表示超时时间，保证出现异常情况下 锁依然能够释放。

3. 避免竞态条件 多个客户端同时尝试获取锁时，**确保只有一个客户端能够成功获取锁**，保证了锁的互斥性
4. redis是分布式缓存，对分布式服务都可见

我的理解：其实本身是一个滑块锁，不是独占锁

生产问题（力求真实）

思考

机器配置：4c8g 一台标准的应用服务器，推荐只部署一个后端应用。

jvm 配置参考；-Xms4096m -Xmx4096m -XX:MaxMetaspaceSize=256m -XX:MetaspaceSize=256m
(小气了，元空间应该512m)



L_I_O

2025-03-09 03:17

小傅哥，我把大营销项目包装成公司的项目，面试被问到系统的qps、tps、总用户数量、日活量、部署服务器数量，这样说可以吗：

平台累计注册用户100万，每次活动1-2万人每天参与。平均RT在200-300ms，Qps日常均值500-1000，巅峰3000-5000，tps600-800，1台lvs，2台nginx，4台应用，2台redis集群模式，2台mysql主从复制，2台es，2台mq，2台普罗米修斯，2台cannal，都是4c8g



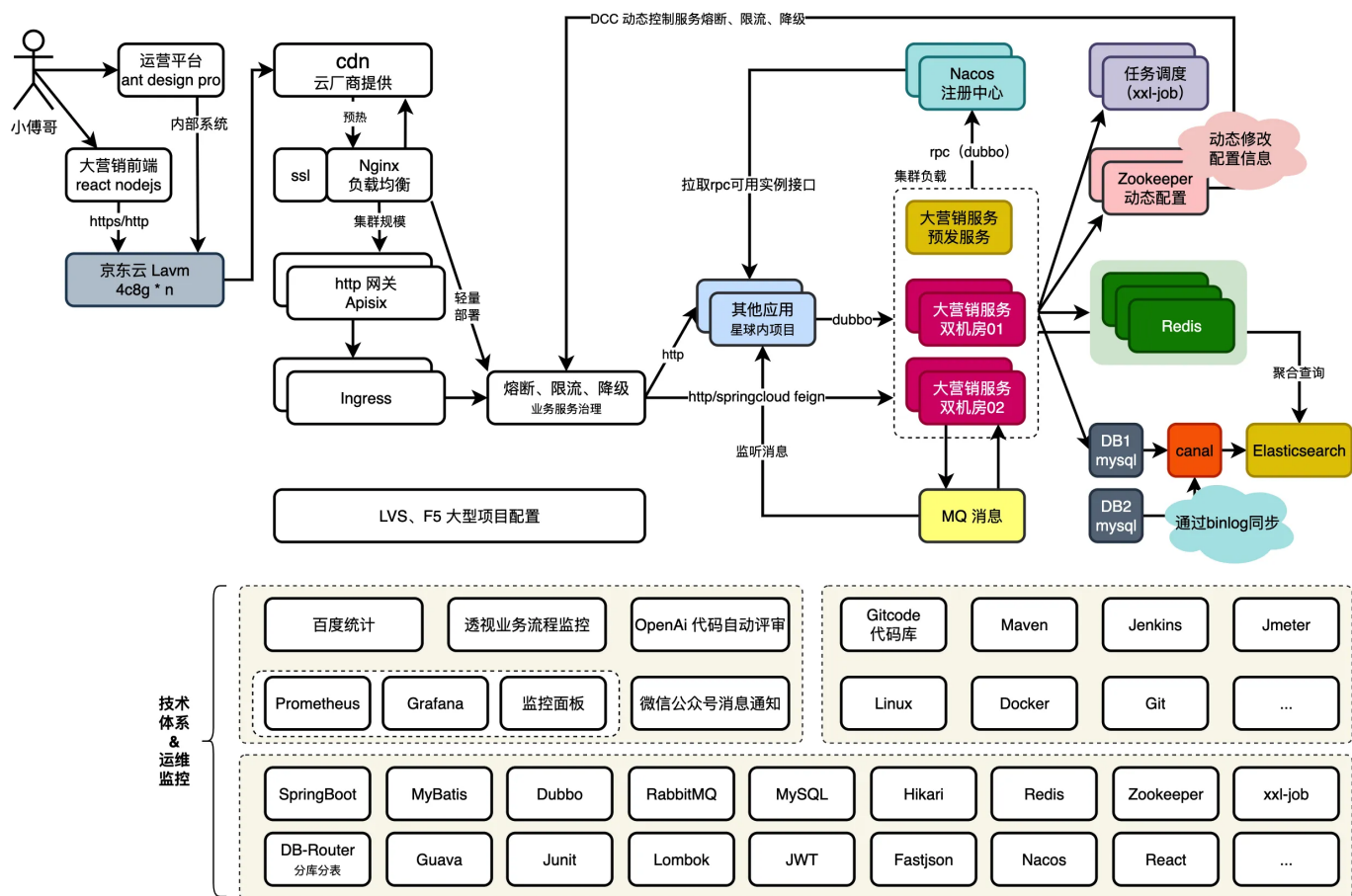
等11人觉得很赞



小傅哥：差不多，可以支撑。之后要自己配置下系统的普罗米修斯监控，压测体验下。

2025-03-09 07:48





1. 像营销这类场景，在push推送、活动预热的情况下，完全可以达到TPS 3000。没问题。
2. 一般不看qps，主要看tps，但tps，你到3000那量就很大了。一般这个tps可以说是整个系统的，之后单抽奖核心接口可以说下具体是多少。之后你按照10个小时算，看看能达到的体量。

我的思考：

用户量基本是 2:8 分，100个用户 20个活跃。需要实际跑压测。

20万活跃用户，每次抽奖2万人参与，一个月5次抽奖活动，数据量不大，提前分库分表。4库64表。

TPS 4*800 3000峰值

平均TPS 1500。

并发用户、RPS、TPS的解读

并发用户：在性能测试工具中，一般称为虚拟用户（Virtual User，简称VU），指的是现实系统中操作业务的用户。

- 说明

并发用户与注册用户、在线用户不同。注册用户一般指的是数据库中存在的用户。在线用户只是“挂”在系统上，对服务器不产生压力。但并发用户一定会对服务器产生压力。

TPS：Transaction Per Second，每秒事务数，是衡量系统性能的一个非常重要的指标。

- 说明

系统每秒处理事务数越多证明您的机器性能越好。

- RPS：Request Per Second，每秒请求数。RPS模式适合用于容量规划和作为限流管控的参考依据。
- RT：Response Time，响应时间，指的是业务从客户端发起到客户端接收的时间。

在性能测试中，通常有两种施压模式：并发模式和RPS模式。传统方式是使用并发用户数来衡量系统的性能（站在客户端视角）。此方法一般适用于一些网页站点的压测（例如H5页面）；而RPS模式主要是为了方便直接衡量系统的吞吐能力TPS而设计的（站在服务端视角），按照被压测端需要达到TPS等量设置相应的RPS，应用场景主要是一些动态的接口API，例如登录、提交订单等等。

简写	英文全称	含义
RT	Response Time	响应时间。通常我们说的响应时间，都是包括了Request Time和Response Time。
HPS	Hits Per Second	每秒点击数
TPS	Transactions Per Second	每秒事务数
QPS	Queries Per Second	在MySQL中指每秒SQL数
RPS	Requests Per Second	每秒请求数
CPS	Codes Per Second	在HTTP协议中，CPS偶有提及，指的是HTTP返回码每秒。
PV	Page View	页面浏览量
UV	Unique Visitor	独立访问者
IP	Internet Protocol	本意是IP地址，在性能中一般指独立IP数
Throughput	/	吞吐量
IOPS	Input/Output Operations Per Second	通常描述磁盘

1. 你的抽奖接口响应时间是多少

200ms （200个线程） 1个线程1S可以处理5个请求（实际可能是4个请求）

单机TPS 800

如果4台机器，可以做到 3000TPS。

2c2g 云服务器，部署项目（含mysql、redis），占用63%内存，抽奖接口响应时间为38~55毫秒

怎么去支撑这样的性能

- 用户抽奖单
- 抽奖流程
 - Redis
 - 责任链
 - 算法
 - 规则树（组合模式）
- 奖品中奖单
- 后续发MQ（事务消息）

分布式一致性的情况： 服务流程编排引擎

2. 这个项目的业务数据，例如用户量、活跃用户量，以及核心业务数据，例如订单系统的下单量等.? 讲述技术维度的数据，例如接口qps，p99值、平均响应时间等

给你一组参考数据；

1. 用户量：180万左右
2. 活跃用户：60万左右
3. 项目抽奖：平均日均40万UV 「集中在活动push区间」
4. 抽奖：qps max 3500~4500 tps 峰值 1200 ~ 1500 从push推送后持续10-15秒 开始逐步递减
5. tp99：110毫秒 tp999 140毫秒 平均响应时间在120毫秒左右。tp999 max rpc抖动 和数据库查询并发较高时会有400毫秒，对于超时250毫秒的rpc会采用重试一次，降低 tp999 最大响应时长，增加系统接口可用率。

数据高了低了也是正常的，在营销活动类中，常常由于运营配置的利益点和push范围决定系统的用户参与量，所以即使可能平常活跃用户不多，但只要有羊毛，就会打到促活的效果。

3. 如果抽奖系统是1500qps，接口响应300ms，然后每秒的并发量是450个，然后面试官问参与人数的时候怎么回答呢？

不要只回答数字，要补充回答，你是如何计算参与人数的？计算过程中遇到什么问题？怎么优化的？

例如：我们注册用户数是xxx万，参与人数在yyy万左右，这是抽奖活动前预估的。整个系统的容量也是按这个的2-4倍设计的。为了得到准确的参与人数，我们利用mysql/redis实现了一个高性能计数器，使用写热点分散的思想提高高并发累加操作的性能。

写热点分散的思想借鉴了LongAddr 的设计。我们也接入了Prometheus和Grafana实现抽奖活动的大屏监控，实时显示qps，参与人数等指标。通过监控大屏，我们发现：参与人数多少，其中，老用户多少，新增注册用户多少，是否在预估的范围内.....

这样，既回答了人数问题，也同时展示了多方面的能力，比如容量设计能力，高并发计数器的设计能力，性能优化的能力，性能监控的能力，项目复盘的能力。里面了也埋了点，供面试官进一步提问。注意，这里的回答要简明扼要，直击重点，但不要展开叙述。等待面试官提出更具体的问题时，再展开。

4. 如果让你来评估项目的QPS的话，你会用什么方式来评估？ (补充：不要做压测，就通过现在的设计以及硬件配置推导QPS应该达到什么水准？)

首先需要根据业务提供的推广规模、渠道、人数，来评估。- 这里前面按照28法则评估过。- 假如系统有1000万用户，那么每天来点击页面的占比20%，也就是200万用户访问。

- 假设平均每个用户点击50次，那么总共有1亿PV
- 一天24个小时，平均活跃时间段算在5个小时内【24*20%】，那么5个小时预计有8000万点击，也就是平均每秒4500个请求。- 4500是一个均值，按照电商类峰值的话，一般是3~4倍均值量，也就是5个小时每秒18000个请求【QPS=1.8万】

不借助工具 采用二八原则，也就是 80%的流量 在20%的时间内产生，首先需要找到系统的评估点：抽奖这个流程，比较复杂 而且还是系统的核心 例如：参加活动抽奖的用户有2000万 在60分钟内完成抽奖 预估有10% 的人会参加活动 那么访问量 = 2000万 * 10% = 200万 因为在60分钟内完成抽奖 所以平均QPS= 200万 / 3600 = 555 一般还会将555 * 2 得到QPS的最大峰值 。这是没有任何数据支撑的二八原则，有数据评估会更准确 接下来就用借助工具 计算单机的QPS 然后决定部署多少台服务器 可以满足 QPS的峰值即可。

5. 抽奖服务量化人群指标是什么？拉新 留存 促活吗 感觉这些数据好难算 面试官问留存率 拉新成本 我都不知道怎么答

- 量化人群：是量化分析师，通过 Python、R 语言对历史数据进行建模（R语言有数据分析公式）。目的是为了根据一个人的，人标签和行为数据，跑出谁喜欢xxx，谁爱购买xxx，谁有xxx意向。之后创建出人群包。这样系统会根据人群包做抽奖、发券等的差异化。

留存率（关键！）

公式：

第N天参与过抽奖的用户中，仍在平台活跃的比例

分层计算：

用户类型	次日留存	7日留存	30日留存
抽奖用户	62%	48%	32%
非抽奖用户	38%	22%	10%

数据支撑：

sql

-- 30日留存率SQL验证

SELECT

```
COUNT(DISTINCT case when is_lottery_user then user_id end) * 1.0 /  
COUNT(DISTINCT user_id) AS retain_rate_30d
```

FROM user_activity

WHERE activity_date BETWEEN '2025-01-01' AND '2025-01-30';

-- 输出：0.32 （32%）

面试话术：

“抽奖用户30日留存达32%，是非参与用户的3.2倍，说明抽奖能有效提升粘性。”

开放式问题

你在做项目中，什么问题难住你的时间最长，为什么？

这是一个开放问题，重点考察你对项目的开发中个人的积累。你可以针对自己的学习过程中，有哪个流程的实现，让你最为有感触，即可回答。

如；可以对大营销抽奖模型流程的设计和库表设计，最为耗时，因为你不断的在思考如何拆解出一个好扩展的松耦合结构，同时拆解后，还要保证搜耦合下的高内聚。所以这部分是比较耗时的。同时也可以说在设定某个方法的，名称、入参、出参时，做了大量的思考。因为名字的定义非常影响以后的理解。好的代码就是文档，所以对于这样的东西花费不少时间。

可以是规则树设计

或者是库存更新的设计

或者四色建模方法实践的一些问题

这个营销系统是之前就有，你去接手了。还是你从0到1 构建的。

如果是公司的项目，有几种情况：

1. 原来有一个抽奖系统，但设计实现上不好满足业务需求，代码维护成本越来越高。所以开始重新架构设计从0开始开发。
2. 进入公司后，刚是公司开始准备在核心业务上做营销拉量的时候，所以从0开始搭建开发。这个过程中做了大量的技术调研和架构设计评审。
3. 进入公司后是一个已经存在的项目，为了更好的支撑业务快速迭代，对系统进行重构。设计了新的模块；规则引擎、策略算法【可以多几种抽奖方式】。

以上按需来描述，并在几种情况里体现出自己的技术项即可。

项目的安全性如何保证？

- 1.鉴权登录「jwt、spring security」，大营销是一个微服务，登录是其他系统。比如 openai 是有登录的，通过微信公众号鉴权。可以参考这个；[网站提示用公众号扫码登录，他们是怎么实现的？ | 小傅哥 bugstack 虫洞栈](#)
2. 数据加密，https、敏感数据 aes 加密【如，openai 的 apikey 可以加密】
3. 防止SQL 注入、入参数据校验 @Valid
4. cors 跨域配置，http请求安全头设置，X-Content-Type-Options、X-Frame-Options、X-XSS-Protection
5. 系统监控；普罗米修斯、在星球的基础教程里有，openai 项目也对接了监控，后续大营销也会对接。还有咱们星球新开的项目，透视业务流程监控。
6. 依赖漏洞扫描，需要一些工具，公司里是提交代码会被扫描。也可以自己搜索工具扫描 owasp dependency-check
7. 代码审计，idea 插件类的，检查代码问题，在星球 idea plugin 教程里有

关于抽奖系统是不是高并发，与传统秒杀系统的区别与联系？

1. 春晚红包正是抽奖的形式，拼多多一进页面就有各种【转转转】来获得一个券，支付完成又一个转转转。直接领券远没有抽奖来的刺激，即使是发券，也是用抽奖方式更多。（故抽奖系统也是高并发场景）
2. 每秒的请求量如果超过1000tps，打到库上资源竞争，都会出现大量的数据库连接等待。一般一个应用分配的数据库连接池也就那么20来个。如果都打到库上，都能把库打挂。

系统的数据怎么判定是否该放入缓存中(redis),判定的具体衡量标准是什么？

1. 有几个指标可以看；访问评率、数据大小、更新频次、生命周期、**热点数据**、**命中率**、生成数据成本（像是大营销预热抽奖概率数据）。
2. 实际业务中综合考量，能提高系统的承载量，吞吐量的都可以添加缓存来处理。以及**缓存+本地缓存**。

QPS与TPS的理解？

QPS（每秒查询数），TPS（每秒事务数）。

QPS 是用来衡量一个系统每秒能处理多少查询请求的指标。它通常用于评估如搜索引擎、数据库系统或Web服务器等高并发场景的性能。

TPS 是用来衡量一个系统每秒能处理多少事务的指标。事务一般是指一组操作的组合，它们要么全部成功，要么全部回滚。TPS更侧重于电商网站、银行等需要事务完整性的应用场景。

大厂面试题

美团】1. 领域模型是怎么设计的，抽奖过程是什么样，DDD四层架构和职责，以及为什么要这么设计？少卖和超卖。

问题：<https://t.zsxq.com/180Qz9WCg>(opens new window)

1. 虽然目前大营销是开发了第1个阶段，但在前面讲解中把所有的流程是都介绍了。领域模型官方话术是头脑风暴，罗列事件和行为，在根据实体来划分领域的。简单说也就是根据业务流程划分的。这个过程包括：活动域、抽奖域、积分域、兑换域。也就是用户通过某种行为记录，发放计算，兑换活动参与资格，完成抽奖获得奖品。
2. 抽奖的流程可以根据目前的设计实现来描述，举例，掘金的这个抽奖就可以用星球的大营销来实现；https://juejin.cn/user/center/lottery?from=lucky_lottery_menu_bar 可以多体验下，体验透彻了，在结合项目的实现，也就能描述出来了。
3. 关于 DDD 的分层结构介绍，和每层的关系；<https://bugstack.cn/md/road-map/mvc2ddd.html>
4. 超卖不会出现，有个保证的点，一个是 decr 值的限制，另外一个是对每个key加锁的兜底设计。确保了不会超卖。**少卖是有可能的**，核心原因是因为 decr 操作和数据操作不是是一个事务，有可能库存扣减完了，**但最终操作库失败了。那么这个库存就丢失了，可能会少卖。**但一般并不会对少卖做过多的流程，如果想管理，也可以把少卖的库存异常，加入单独的 redis 队列来重新消费就可以了。

```
1  public Boolean subtractionAwardStock(String cacheKey, Date endDateTim  
   e) {  
2      long surplus = redisService.decr(cacheKey);  
3      if (surplus < 0) {  
4          // 库存小于0, 恢复为0个  
5          redisService.setAtomicLong(cacheKey, 0);  
6          return false;  
7      }  
8      // 1. 按照cacheKey decr 后的值, 如 99、98、97 和 key 组成为库存锁的key进  
   行使用。  
9      // 2. 加锁为了兜底, 如果后续有恢复库存, 手动处理等, 也不会超卖。因为所有的可用  
   库存key, 都被加锁了。  
10     String lockKey = cacheKey + Constants.UNDERLINE + surplus;  
11     Boolean lock = false;  
12     if (null != endDateTime) {  
13         long expireMillis = endDateTime.getTime() - System.currentTimeMillis() + TimeUnit.DAYS.toMillis(1);  
14         lock = redisService.setNx(lockKey, expireMillis, TimeUnit.MILL  
   ISECONDS);  
15     } else {  
16         lock = redisService.setNx(lockKey);  
17     }  
18     if (!lock) {  
19         log.info("策略奖品库存加锁失败 {}", lockKey);  
20     }  
21     return lock;  
22 }
```

库存扣减:

```

1      log.info("规则过滤-库存扣减 userId:{} strategyId:{} awardId:{})", userI
      d, strategyId, awardId);
2      // 扣减库存
3      Boolean status = strategyDispatch.subtractionAwardStock(strategyI
      d, awardId, endDateTime);
4      // true; 库存扣减成功, TAKE_OVER 规则节点接管, 返回奖品ID, 奖品规则配置
5      if (status) {
6          log.info("规则过滤-库存扣减-成功 userId:{} strategyId:{} awardId:
          {}\"", userId, strategyId, awardId);
7
8          // 写入延迟队列, 延迟消费更新数据库记录。【在trigger的job; UpdateAwar
          dStockJob 下消费队列, 更新数据库记录】
9          strategyRepository.awardStockConsumeSendQueue(StrategyAwardSto
          ckKeyV0.builder()
10              .strategyId(strategyId)
11              .awardId(awardId)
12              .build());
13
14          // 注意; 根据数据库表中配置走不同的节点。目前数据库配置 ALLOW 是走到下一
          个节点。
15          return DefaultTreeFactory.TreeActionEntity.builder()
16              .ruleLogicCheckType(RuleLogicCheckTypeV0.TAKE_OVER)
17              .strategyAwardV0(DefaultTreeFactory.StrategyAwardV0.bu
          ilder()
18                  .awardId(awardId)
19                  .awardRuleValue(ruleValue)
20                  .build())
21              .build();
22      }
23
24      // 如果库存不足, 则直接返回放行
25      log.warn("规则过滤-库存扣减-告警, 库存不足.userId:{} strategyId:{} awa
          rdId:{})", userId, strategyId, awardId);
26      return DefaultTreeFactory.TreeActionEntity.builder()
27          .ruleLogicCheckType(RuleLogicCheckTypeV0.ALLOW)
28          .build();

```

【淘天】2. 对于数据库和redis的一致性怎么解决(关键)

技术方案：

1. Redis decr扣减库存
2. 通过MQ消息异步去更新数据库中的奖品库存（最终一致性）

3. 活动到期后，任务扫描活动产生订单量，校准库存

问题：<https://t.zsxq.com/18ddtmzqW>(opens new window)

- 第1个手段；每次消费 decr 值，写入延迟队列，趋势更新数据库数据。最终一致。
- 第2个手段；decr 库存值消耗为0时，发送mq消息，更新最终库存量。（可能不准，比如 decr 值消耗中，少卖的情况）
- 第3个手段；活动到期后，任务扫描活动产生订单量，校准库存。

【其他】3. 什么情况下使用DDD架构，什么情况下使用mvc架构？

DDD 是软件设计方法，对复杂的项目更为合适。但这里要清楚，DDD 如果只的是设计方法中的建模工程结构，和 MVC 对比的话。DDD 的结构更为先进，即使不使用 DDD 的软件设计方法，只遵循这套结构，都是可以编写出非常好的代码的。MVC 的约束相对较低，个人开发还好，如果多人协作，会出现腐化严重的问题。

【其他】4. 设计模式带来了什么好处？举个例子

设计模式可以让工程设计的迭代性、扩展性、维护性，都更强，更好。如，抽奖策略计算中，用到了责任链、组合模式的规则树。规则树可以动态的调整配置的节点，来满足各种业务诉求。还可以结合产品需求，迭代的时候添加对应的节点开发就可以。避免了大量if...else的使用，让变动范围缩小到指定的类中。研发成本更低，提测质量更高（测试更快），交付效率更强。这些都是使用设计模式的优势。

【其他】5. 最近大营销抽奖项目被问到抽奖项目按道理应该不是一个高并发的过程，不是想那种抢优惠券这种属于高并发，那如果抽奖不是高并发，那为啥还要把库存缓存到redis去抗并发呢？

要好好留心日常的场景，都是面试的话术；

1. 春晚红包是不是抽奖，拼多多一进页面就有各种【转转转】来获得一个券，支付完成又一个转转转。直接领券远没有抽奖来的刺激，即使是发券，也是用抽奖方式更多。
2. 每秒的请求量如果超过1000tps，打到库上资源竞争，都会出现大量的数据库连接等待。一般一个应用分配的数据库连接池也就那么20来个。如果都打到库上，都能把库打挂。
3. 面试这么问，大部分是为了通过一个质疑的场景，来看是否有思考过。而不是面试问什么就是什么，反而是问什么就不是什么，但不是什么，要拿出举证理由。

【其他】6. 抽奖算法提问

1. 问：数据库路由算法 答：hashcode保证两次散列结果一致，扰动函数保证散列均匀。 问：除了hashcode数据库路由算法还有其他的吗？
 2. 问：抽奖除了加分布式锁还有没有别的思路？ 答：这里本质上还是保证库存不超卖，说了低并发下的数据库行锁，redis队列 问：还有其他思路吗？ 答：想不出来了（面试官说依赖高性能的中间态，这块没理解）
 3. 计次模型，设计一天一次的参与规则，怎么实现？
 4. 抽奖算法的实现？这个随机怎么可控，怎么避免高价的奖品一开始就被抽掉了，类似于活动中间阶段才能中一次一等奖
 5. 了解过其他抽奖算法吗？
 6. 黑名单如果上到一定规模，比如百万级别，有其他的设计思路吗？
 7. 扣减库存的分段竞态锁用 incr 还是 decr，为什么是 incr/decr？
 8. 如果活动开始了，要加奖品库存怎么办？
- 问题1； [https://bugstack.cn/md/algorithm/logic/math/2022-11-05-fibonacci.html\(opens new window\)](https://bugstack.cn/md/algorithm/logic/math/2022-11-05-fibonacci.html(opens new window))
 - 问题2；还有，预生成中奖结果。比如微信春晚红包，提前把奖品随机好，写入到个人记录。用户开奖只是到点查看结果。【如果提到高性能中间件这个可以再继续让面试聊下，因为有可能问题和听者理解的不一致了】
 - 问题3；这个课程里提供了总、月、日库存。
 - 问题4；有次数锁、有权重，可以避免一开始就中一等奖。也是课程的。
 - 问题5；算法如；线性同余生成器、梅森旋转算法、洗牌算法、加权随机。
 - 问题6；黑名单一般会设定人群标签，写入到 Redis 使用。之后库里只是配置了一个人群标签的标识。
 - 问题7；incr 和总量比，decr 和 0 比。decr 适合过程中不允许补充库存的。incr 可以在过程中添加库存，因为总量可以增加对比。【问题8】

【携程】7. 我跟他说用户在抽奖系统用积分兑换抽奖机会，我方会向mq发消息，积分微服务那边（不是我们组负责的）拿到消息之后扣减会员的积分，他问我：

- 你为什么在这种积分扣减用mq？我说流量削峰，他说这不是关键。
- 如果用户用脚本频繁兑换抽奖机会，你们怎么应对的。

- 果下游服务迟迟没有对mq进行消息消费，你们怎么处理的？你假设作为系统的架构者，是怎么监控这种状况的。

解答：[https://t.zsxq.com/0jjqr\(opens new window\)](https://t.zsxq.com/0jjqr(opens new window))

1. 当你描述为积分和抽奖是单独的系统，你要生成兑换商品订单，拿订单的唯一单号，rpc/http 调用积分系统扣减积分，返回一个交易单号和时间。【如果是微信、支付宝类支付，就需要等待回调，因为还要走支付渠道很多流程，没法直接返回】
2. 你通过积分支付接口返回的信息开始变更订单记录，发放抽奖次数。
3. 用户是用自己的积分兑换的，账户会进行额度拦截。之后接口会配置频次拦截【大营销第4阶段有这个设计】
4. 系统是有边界管控的，在实际工作中，当前的系统要保证发送mq，下游的系统要保证消费mq。mq的消费会有监控配置，比如日常每分钟100次，如果连续n次在n分钟内，低于80次或者高于140次，则进行报警。这样就可以监控到了。

【其他】8. 关于上下游交互的问题

1. 每次面试官都会问一嘴，基于ratelimiter是做单机限流吗？实际生产中主要是使用分布式限流吗，为什么要基于分布式做呢，单机限流满足不了需求吗，分布式限流具体实现方式是什么呢，基于redis、zookeeper自己实现还是基于限流框架做呢
2. 限流、降级有必要做到外部服务粒度的嘛？（例如有3个外部系统，有必要为每个系统单独限流、降级吗，例如只是不提供给某个系统服务）
3. 奖品单状态的变更，我还是有一点模糊，如果是下游发奖，那下游应该是不能访问奖品服务的库吧，那下游发完奖如何更新奖品单状态呢？会有回调接口或者是下游再发一个发奖完毕的消息吗？
4. 抽奖活动会和业务系统绑定吗？是不是得在活动表加一个字段，然后鉴权的时候只能调用当前系统的抽奖活动。

解答：[https://t.zsxq.com/7uTKz\(opens new window\)](https://t.zsxq.com/7uTKz(opens new window))

1. 基本都是做单机限流，要的就是单台机器承载量。分布式做集中限流，那么瓶颈就会出现在限流这块了。
2. 如果外部服务是那种付费购买的调用权益就有必要，例如你去调用一些学生认证、公安网，不同的权限调用频次是不一样的。
3. 内部系统一般是发消息，外部系统一般是回调。
4. 会配置渠道方，不同的渠道方都可以加授权。

【网易】9. 在抽奖过程时，需要预热处理，可是不可能让用户点击活动装配呀，这个问题你怎么解决？

第一个视频里提到了，是在活动运营配置后，可以审核通过就预热了。等待活动到了有效期就可以使用了。

【网易】10. 在抽奖的结果中，如果一个用户在前端抽到了一个奖品，对用户是可见的。可是在扣减库存时，却发现没有了对应的库存，此时应该怎么办？

库表设计中，抽奖的每条策略是有库存限制的，抽到后才会展示给用户。如果库存不足直接就是兜底积分了。这个也有人遇到过反过来问的，为什么要给策略上也加库存，直接奖品加库存不就可以。其实是不可以的。

【其他】11. 既然为了想承受更高并发使用redis做库存扣减，那生成奖品id后续要等中奖订单入库才能给用户展示结果吗？那用redis做库存扣减不就没意义了

最核心的其实是，如果没有redis做库存扣减，就要数据库里的一个库存记录做扣减。那么就会有成千上万条的请求在同一个表行记录开始独占竞争加锁，其余的请求进入等待，直至耗尽所有数据库连接。那么整个系统服务也就会被拖垮，一个普通的查询也会从原来的几十毫秒变成到一分钟也拿不到结果了。

使用redis，就是为了解决这个事情。之后在写入库里的记录，都是无竞争的。那么就不会让数据库被夯住。可以快速被处理。在大厂的数据库配置，基本这类操作不会被 redis 慢多少。

解答：<https://t.zsxq.com/kCjDL>(opens new window)

抽奖流程

1. 校验
2. 创建抽检单
3. 抽奖责任链
 - a. 黑名单 过滤
 - b. 随机抽奖
 - c. 权重抽奖
 - d. 抽了10次必中
4. 抽奖规则树
 - a. 校验奖品库存
 - b. 扣减Redis中的奖品库存
 - i. 发送MQ消息 异步扣减数据库库存（有一个唯一ID，保证MQ消费的幂等性）
5. 发奖前操作

- a. 写入用户奖品单（写入后用户才能看到，也是写数据库 不是更新Redis）
- b. 更新抽奖单状态（已使用）

Redis扣减成功，数据库写入中奖记录和更改抽奖订单状态为已使用，这时候数据库写入又没有热点更新很快的，用Redis扣减库存是因为数据库扣减库存存在热点更新问题而且数据库也抗不了这么大的并发，况且数据库也不止扣减库存这一个表，还有别的表，不能影响别的业务正常执行。

【其他】12. 你说你的系统压测指标是3000QPS（简历上我写了3000QPS压测指标），你认为你做的是这个系统压测到3000的时候，他的瓶颈在哪里？

1. 你应该是描述的单个接口压测，3000qps，qps 指查询，这个过程没有写库，没有事务的操作。瓶颈往往在于接口串联的多，也包括rpc微服务，层层调用带来的性能损耗。其中**性能损耗最大的，一般是风控类接口**。之后优化可以用多线程并行调用，之后在统一把数据合并处理。你在回答的时候，可以讲，之前是多少qps，响应时间tp99、tp999、max是多少，之后经过与外部接口的协调，增加redis缓存预热，以及你在把接口通过多线程异步并行查询，提高了接口性能，增加了qps指标。另外，还有是单机还是分布式，每台机器配置的jvm，比如原来的指标配置是什么样，这次配置的是多少（这些可以从普罗米修斯监控得出，之后整理成回答）**如果回答数据类的，一定要压测。**
 2. 抽奖用的是你的个人库存，这部分可以加分布式锁，一个活动，一个sc场景，只能一个抽奖进行中活动，这样组合一个key加分布式锁就可以了。
 3. incr 这个星球里有很多资料了，分段锁接近于无锁化，乐观锁。而以前的直接加锁，锁全活动是独占锁，悲观锁。我们要的目的就是接近于不加锁，并且保证性能。另外 lua 脚本实际压测性能并不好。incr + 锁，尽可能避免超卖。但不加锁，基本等于裸奔，虽然大概率不出问题，但出问题是不可知的。我们做系统设计，就要考虑它可能存在的风险，尽可能降低，而不是等待它发生。资料：<https://t.zsxq.com/F1J0L> – 这里的话术可以使用「海恩法则、薛定谔的猫、墨菲定律、蝴蝶效应及熵增原理」。
- 解答：<https://t.zsxq.com/Gdttm>(opens new window)
 - 扩展：<https://t.zsxq.com/FWBzl>(opens new window)

扣库存（最难点）

扣库存：加就和总量比，减就和 0 比

1. 完整搞懂 扣库存 和 补库存的逻辑

a. 扣库存:

i. redis decr

ii. redis setnx 分段锁 锁被扣掉后的库存数字(100 -> 99 锁住 99 不能再从100扣减到 99)

b. 补库存:

2. 扣库存为什么要这么扣

扣库存

Java

```
1 public boolean subtractionActivitySkuStock(Long sku, String cacheKey,
2     Date endDateTime) {
3     long surplus = redisService.decr(cacheKey);
4     if (surplus < 0) {
5         // 库存小于0, 恢复为0个
6         redisService.setAtomicLong(cacheKey, 0);
7         return false;
8     }
9     // 1. 按照cacheKey decr 后的值, 如 99、98、97 和 key 组成为库存锁的key进
    行使用。
10    // 2. 加锁为了兜底, 如果后续有恢复库存, 手动处理等【运营是人来操作, 会有这种情
    况发放, 系统要做防护】, 也不会超卖。因为所有的可用库存key, 都被加锁了。
11    // 3. 设置加锁时间为活动到期 + 延迟1天
12    String lockKey = cacheKey + Constants.UNDERLINE + surplus;
13    long expireMillis = endDateTime.getTime() - System.currentTimeMillis() + TimeUnit.DAYS.toMillis(1);
14    Boolean lock = redisService.setNx(lockKey, expireMillis, TimeUnit.
    MILLISECONDS);
15    if (!lock) {
16        log.info("活动sku库存加锁失败 {}", lockKey);
17    }
18
19    if (surplus == 0) {
20        // 库存消耗没了以后, 发送MQ消息, 更新数据库库存
21        eventPublisher.publish(activitySkuStockZeroMessageEvent.topic
    (), activitySkuStockZeroMessageEvent.buildEventMessage(sku));
22    }
23
24    return lock;
25 }
```

- 明确一个问题： 库存 > 0 才能扣减，因此我们先扣减，如果小于0，那么直接返回失败（库存不足了）

问题1：加分段锁的目的是为了什么？

答：核心目标是为了防止低概率下集群、主从故障导致的超卖，做一个兜底逻辑。

问题2：如果不需要补库存，分段锁是不是可以不用加？

答：即使不考虑手动补库存的情况，如果集群、主从故障，不加分段锁还是会可能超卖的，所以这里分段锁不单单是为了补库存的场景而设计。

问题3：那直接 incr 不可以吗？

答：在 redis 集群模式下，incr 请求操作可能发生网络抖动超时返回。这个时候 incr 有可能成功，也有可能失败。可能是请求超时，也可能是请求完的应答超时。那么 incr 的值可能就不准。【实际使用中10万次，可能会有10万零1和不足10万】，那么为了这样一个临界状态的可靠性，所以添加 setNx 加锁只有成功和失败。

还有一种情况是主从切换的时候，如果主节点的 incr 还没同步到从节点，主节点挂了，丢失了部分未同步的数据，incr 的值从 8 变成 6，如果没有加锁就可能超卖，属于极端情况下的一种兜底策略，有 setNX 锁拦截后，会更加可靠。

问题4：那如果考虑集群故障，机器挂掉的情况，setNX 不也会报错吗？

答：setNX 如果失败了，就直接报错返回 "活动库存不足" 即可，也就是可能会导致少卖，但是不会导致超卖。并且 incr 和 setNX 的 key 不同，incr 的 key 和滑块锁的 key 大概率不在同一节点上，从而双重保证，如果 setNX 的 key 和库存的 key 节点都 down 机了，那这里确实有超卖的可能，不过这个概率可以低到忽略不计。

问题5：decr 和 incr 两种扣减方式有什么不同？

答：二种方式都可以，decr 适合固定库存场景，和 0 对比，incr 适合可以补库存的场景，和库存总量对比。

问题6：那为什么要分段，直接对一个 key setNX 不可以吗？

答：分段锁的话，setNX 因为是非独占锁，所以 key 不存在释放。setNX 的 key 的过期时间可以优化为活动的有效期时间为结束。而独占锁，其实你永远也不好把握释放时间，因为秒杀都是瞬态的，释放的晚了活动用户都走了，释放的早了，流程可能还没处理完。

问题7：incr 扣减模式下，如果同一个用户并发进来，那么缓存中的库存就会+并发数，但实际这个用户只会领取到一条数据，所以就要恢复并发数-1的库存数量。这样种情况并不是 redis 不稳定导致的，而是同一用户并发导致的，应该及时去恢复数据啊，不然的话缓存中的库存直接一下就给一个用户并发干没了，然后再去恢复，效率太低了吧？（幂等性设计）

答：不需要恢复，还是回到上面，核心是保证不超卖，关于库存恢复，一般这类抽奖都是瞬态的，且 redis 集群非常稳定。所以很少有需要恢复库存，如果需要恢复库存，那么是把失败的秒杀 incr 对应的值的 key，加入到待消费队列中。等整体库存消耗后，开始消耗队列库存，等补偿恢复，活动已经基本过去了。所以超卖，快速结束是最好的。这个一般是基于运营策略配置何种方式恢复库存，可以失败的专门扫描到恢复库存列表用于消耗，也可以不恢复（因为失败概率很低，也允许不超买即可）。

今天在面试的时候，大营销项目被狠狠的拷打了（链接如下）



知识星球 | 深度连接铁杆粉丝，运营高品质社群，知识变现的工具

知识星球是创作者连接铁杆粉丝，实现知识变现的工具。任何从事创作或艺术的人，例如艺术家、工匠...
深度连接铁杆粉丝，运营高品质社群，知识变现的工具

主要有以下问题：

1. 再细化一下，多个用户过来请求库存值，是不做任何加锁操作直接读取出来对吗？对。追问：decr已经是原子操作了「多个请求一定是串行的」，你为什么还要加setnx锁？「是否有必要加这个锁」

- 我的回答：考虑网络异常，库存恢复，
- 面试官：假设不考虑网络问题，那你加锁是要解决那种场景问题？
- 面试官--->加锁的意义是什么，不加这个锁又会出现什么问题？

2. 假设我当前活动奖品A还要十个库存，运营想再加十个库存，你这个时候是怎么处理的，支持吗？
追问：假设我再增加一次库存，或者n次库存，你是怎么处理的呢？「运营经常对库存做一些操作」

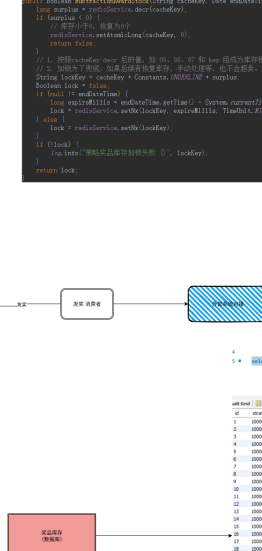
- 我的回答：活动降级补充库存？或者配置多个库存池，或者用incr+消息队列去补充库存
- 面试官---> 你们这块是没太考虑实现吗？，项目能启动，但缺少了维护的考虑

使用的是decr进行的库存扣减，然后setnx(decr扣减后的值)，锁🔒 不过期的情况下，假如已经消费到0了，那么你直接对该奖品库存热key更新之后，原先存在的setnx会不会锁住一部分库存，导致走兜底奖励(就相当于库存添加失败。

1. 其实本身原本的库存，扣减最早的方式是数据库行级锁，数据库扛不住。优化到redis缓存，使用了redis独占锁，但这样会出现排队问题，导致有库存，但吞吐量不佳。
 2. 后来设计为颗粒度更细的无锁化（乐观锁）设计。incr/decr 是原子操作，基本也不会出问题，这个操作就有点像数据库计数，1、2、3、4... 而加锁类似写了流水记录。相对来说这样的方式更可靠。避免在：集群配置、网络、库存恢复、人工调整等场景时，导致 incr/decr 的值不对。因为如果不加锁，这个时候不对，是不知道的。不知道就很可怕，哪怕有万分之一的概率，只要有足够多的用户参与，也会出问题。（尤其运营配置，运营是很多人，也有很多新人，实际公司里很多事故都是运营配置导致的，而且出问题的时候是一堆人围着一个运营，运营很慌，配置错的概率更大）
 3. 可以先调整数据库库存，之后使用 incrby 给缓存加库存。这个时候要注意使用 incr 和总量对比。如果有对库存的失败记录，可以用 incr 和 总量 + 恢复了对比。
- 加锁，用于兜底；集群配置、网络、库存恢复、人工调整等。尽量避免出问题，加库存就是其中的一个。

```

graph TD
    A([订货]) -- 处理订单 --> B[库存货品]
  
```

[illegible]